Optimising Multi-Scalar Multiplication: A Comparative Analysis of Bos-Coster, Bucketing, and Pippenger's Algorithms

by

YOUSEF HESHAM MOSAAD ABDELGHANI

URN: 6738880

A dissertation submitted in partial fulfilment of the

requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2025

Department of Computer Science

University of Surrey

Guildford GU2 7XH

Supervised by: Dr. Robert Granger

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Yousef Hesham Mosaad Abdelghani

May 2025

c Copyright Yousef Hesham Mosaad Abdelghani, May 2025

Abstract

This research provides a comprehensive comparison of three key algorithms for multi-scalar multiplication; a fundamental operation in cryptographic systems that impacts the performance of digital signatures, key exchange protocols, and zero-knowledge proofs. The study evaluates the Bos-Coster algorithm, Bucketing algorithm, and Pippenger's algorithm through both theoretical analysis and empirical testing to determine their optimal use cases in practical cryptographic applications.

The research implements multiple variants of each algorithm and evaluates them using a systematic benchmarking framework that measures execution time, operation counts, and memory usage across varying input sizes and scalar bit lengths. This approach bridges the gap between theoretical predictions and practical implementation considerations, providing specific guidance for algorithm selection in real-world scenarios.

Results reveal distinct performance regions for each algorithm: Bos-Coster performs exceptionally well for small inputs (5-20 points) with moderate bit lengths, making it ideal for constrained environments. Comb-Bucketing excels with medium-sized inputs (20-50 points) across all bit lengths, offering a good balance between simplicity and efficiency. Pippenger's algorithm demonstrates superior performance for large inputs (50+ points) with large scalar bit lengths (128+ bits), confirming its theoretical asymptotic advantages in practical implementations.

The study identifies specific crossover points where algorithm preferences should shift, providing concrete thresholds that can guide implementation choices in cryptographic systems. Additionally, the research analyses memory usage patterns, revealing that Pippenger's algorithms, while computationally efficient, require more memory than alternatives, an important consideration for memory constrained environments.

These findings enable developers to make informed decisions when selecting multi-scalar multiplication algorithms for specific cryptographic contexts, potentially enhancing the efficiency of security systems ranging from resource constrained Internet of Things devices to high performance cryptography servers, while maintaining the necessary security properties.

Acknowledgements

I would like to thank my project supervisor for introducing me to pippenger’s algorithm and providing guidance during this dissertation from the very start, to that I am grateful. I would also like to dedicate this dissertation to my mother for always supporting me.

Table of Contents

List of Figures 9

List of Tables 15

1 Introduction 25

1.1 Background and Motivation 25

1.2 Problem Statement 25

1.3 Challenges and Complexity 26

1.4 Approach 26

1.5 Project Aims and Objectives 27

2 Literature Review 28

2.1 Evolution of Multi-Exponentiation Algorithms 28

2.2 Fundamental Concepts and Early Developments 28

2.3 Pippenger's Algorithmic Framework 28

2.4 The Bos-Coster Approach 29

2.5 Bucketing Algorithms and Straus's Method 30

2.6 Comparative Analysis, Performance Characteristics and Trade-offs 30

2.7 Applications in Modern Cryptography 31

2.8 Recent Developments and Open Challenges 31

2.9 Summary and Research Direction 32

3 Methodology and Implementation 33

3.1 Set-up 33

3.1.1 Problem Definition and Requirements Analysis 33

3.1.2 Elliptic Curve Environment Configuration 33

3.1.3 Test Case Generation 34

3.1.4 Performance Measurement Infrastructure 34

3.1.5 Technical Challenges and Design Decisions 34

3.2 Bos-Coster Algorithm 35

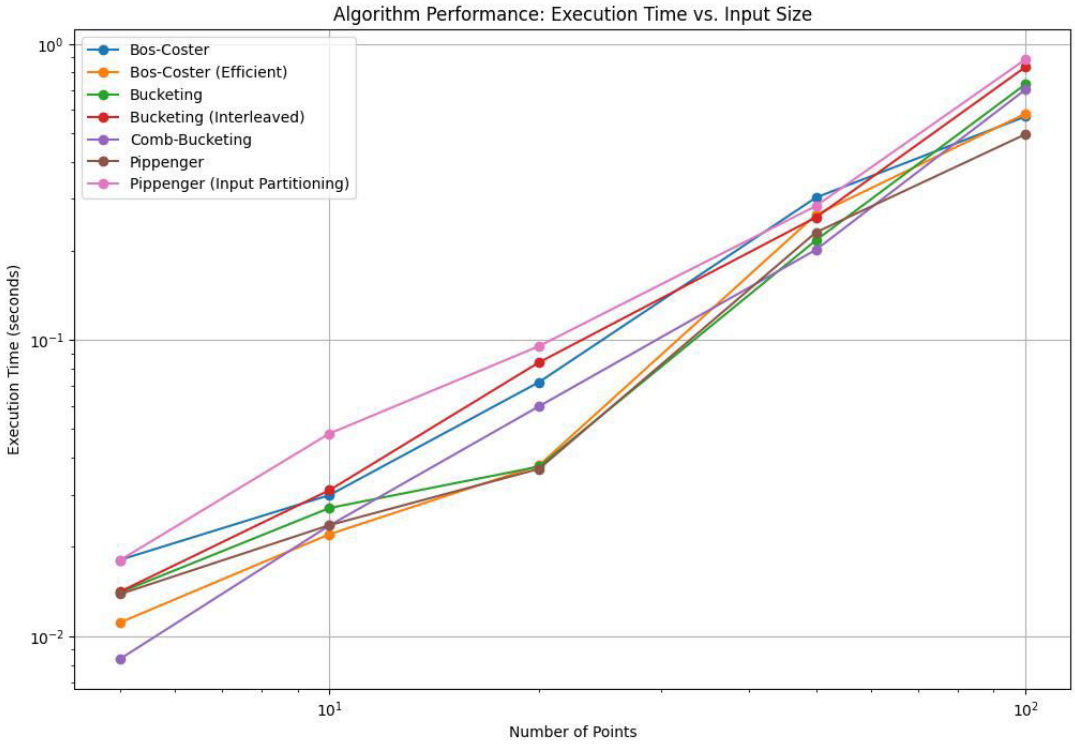
3.2.1 Theoretical Foundations 35

3.2.2 Implementation Strategy 36

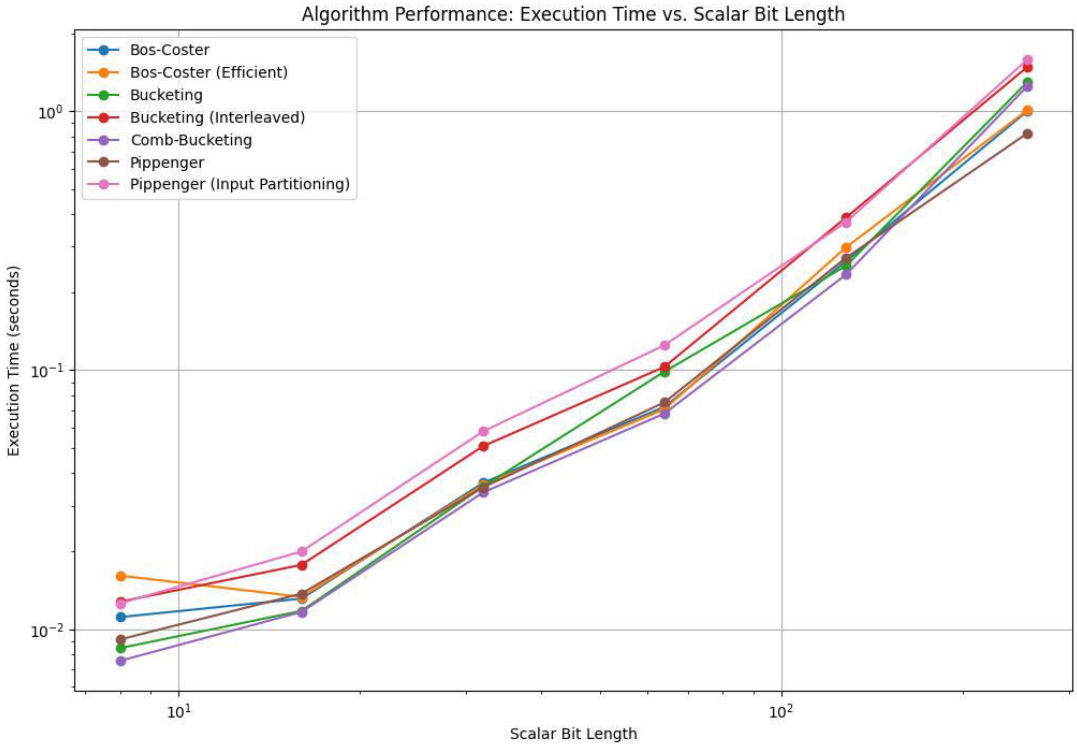
3.2.3 Design Considerations and Optimisations 36
3.2.4 Algorithmic Complexity Analysis 37
3.2.5 Implementation Challenges 37
3.2.6 Testing and Validation 38
3.3 Bucketing Algorithm 39
3.3.1 Theoretical Foundations 39
3.3.2 Implementation Strategy 39
3.3.3 Design Considerations and Optimisations 40
3.3.4 Algorithmic Complexity Analysis 41
3.3.5 Implementation Challenges 41
3.3.6 Testing and Validation 42
3.4 Pippenger's Algorithm 43
3.4.1 Theoretical Foundations 43
3.4.2 Implementation Strategy 44
3.4.3 Design Considerations and Optimisations 44
3.4.4 Algorithmic Complexity Analysis 45
3.4.5 Implementation Challenges 46
3.4.6 Testing and Validation 47
3.4.7 Refinements 48
3.5 Comparison Framework 48
3.5.1 Design Considerations and Requirements Analysis 48
3.5.2 Measurement Methodology 49
3.5.3 Test Case Generation 50
3.5.4 Benchmark Implementation 51
3.5.5 Data Analysis and Visualisation 51
3.5.6 Implementation Challenges 52
3.5.7 Comprehensive Evaluation Process 53
3.5.8 Validation Against Theoretical Predictions 53
4 Results and Evaluation 55

4.1 Performance Analysis Framework 55
4.2 Execution Time Analysis 55
4.3 Operation Count Analysis 56
4.4 Operation Breakdown Analysis 57
4.5 Operations per Point Analysis 58
4.6 Comparative Analysis 58
4.7 Limitations and Challenges 61
5 Legal, Social, Ethical and Professional (LSEP) 62
5.1 Future Implications for Cryptographic Applications 62
5.2 United Nations Sustainable Development Goals. 62
6 Conclusion 64
6.1 Future Work 64
7 References 66

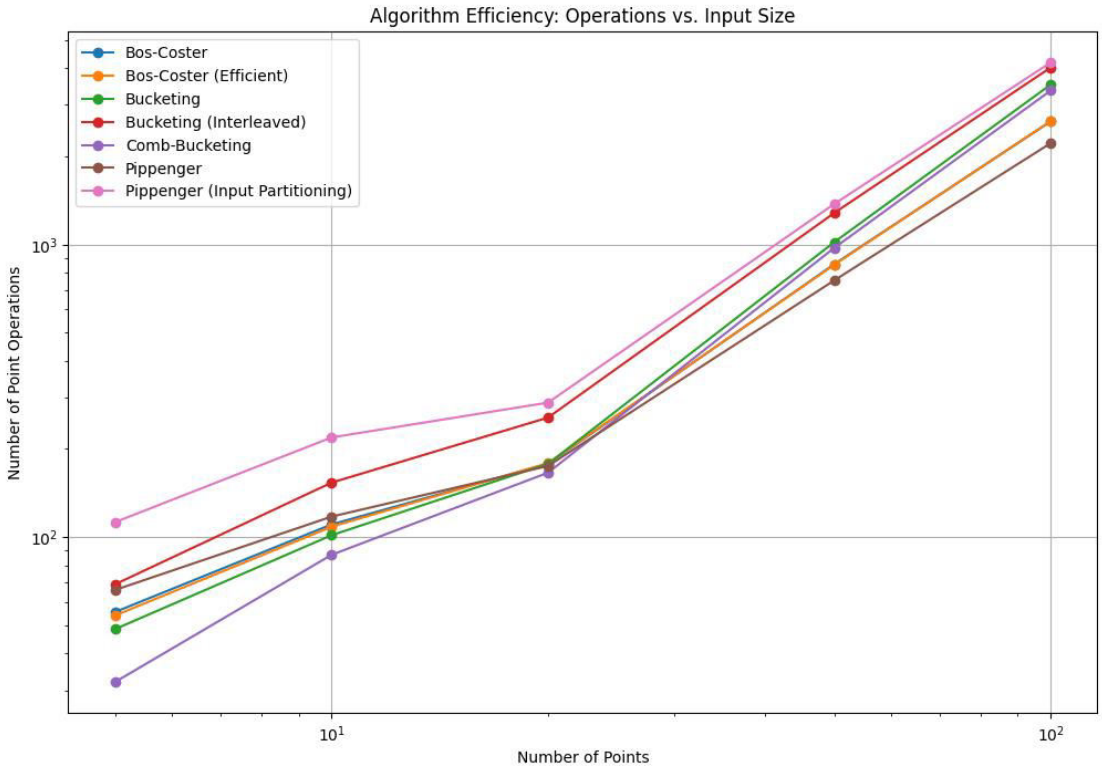
List of Figures



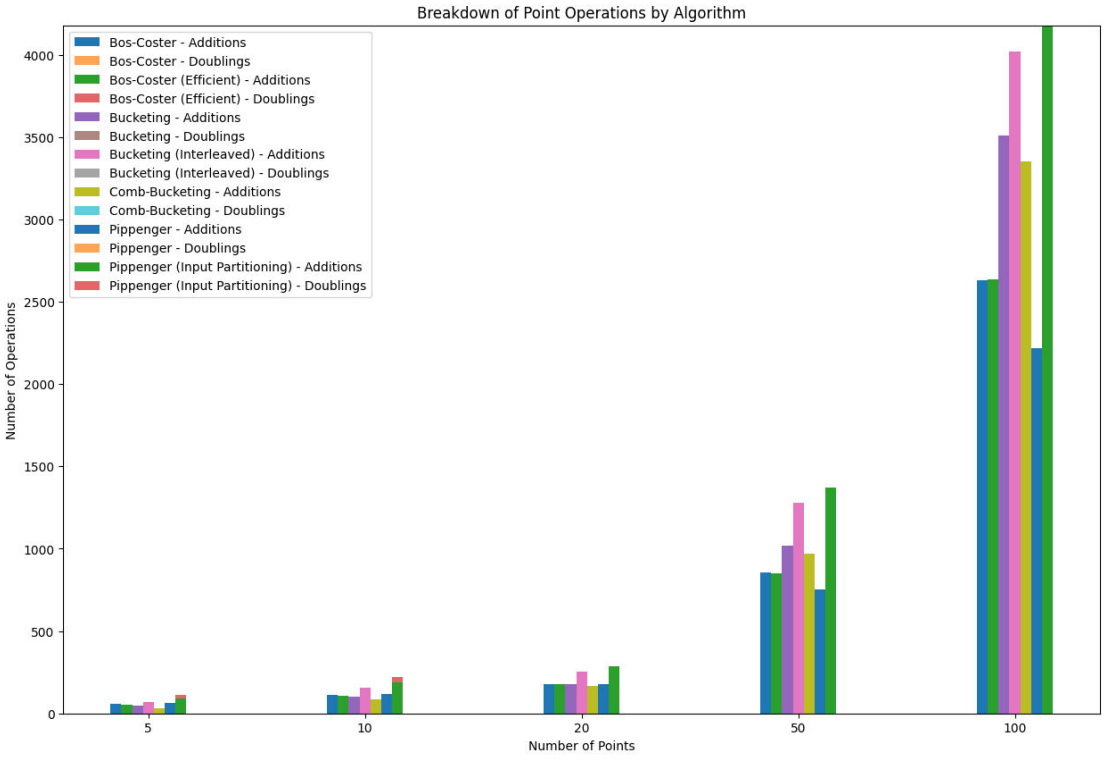
(figure 1)



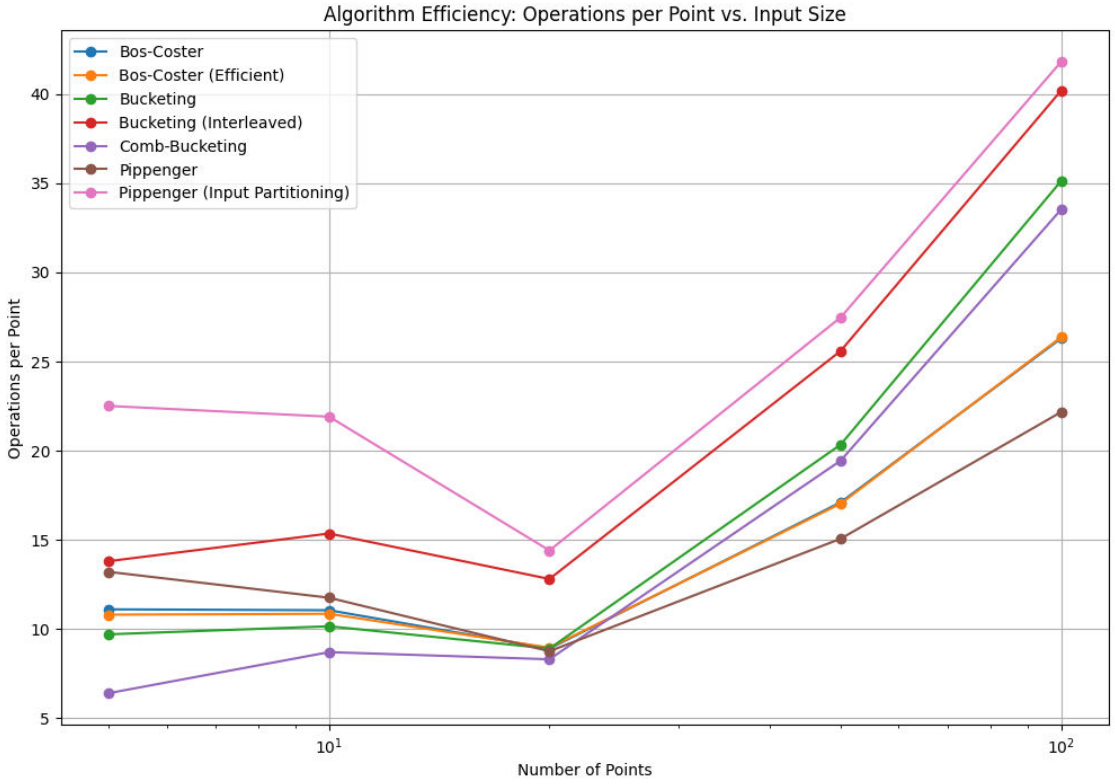
(figure 2)



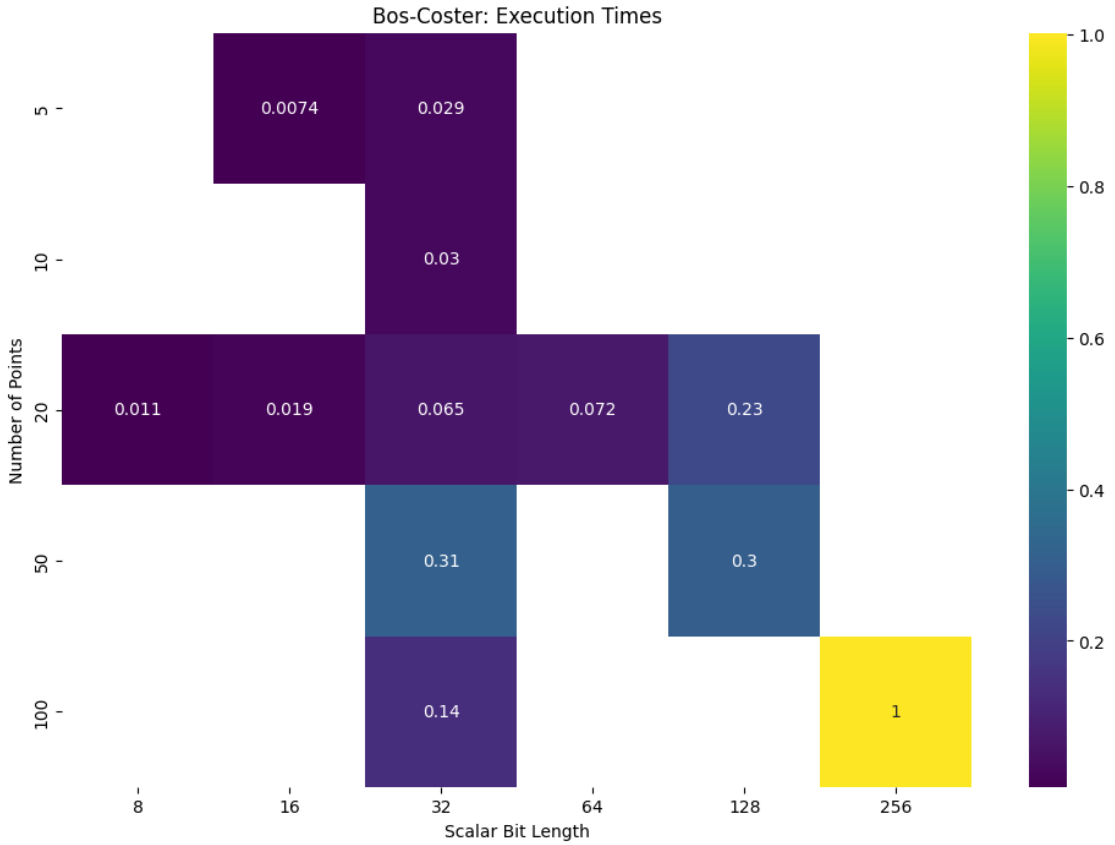
(figure 3)



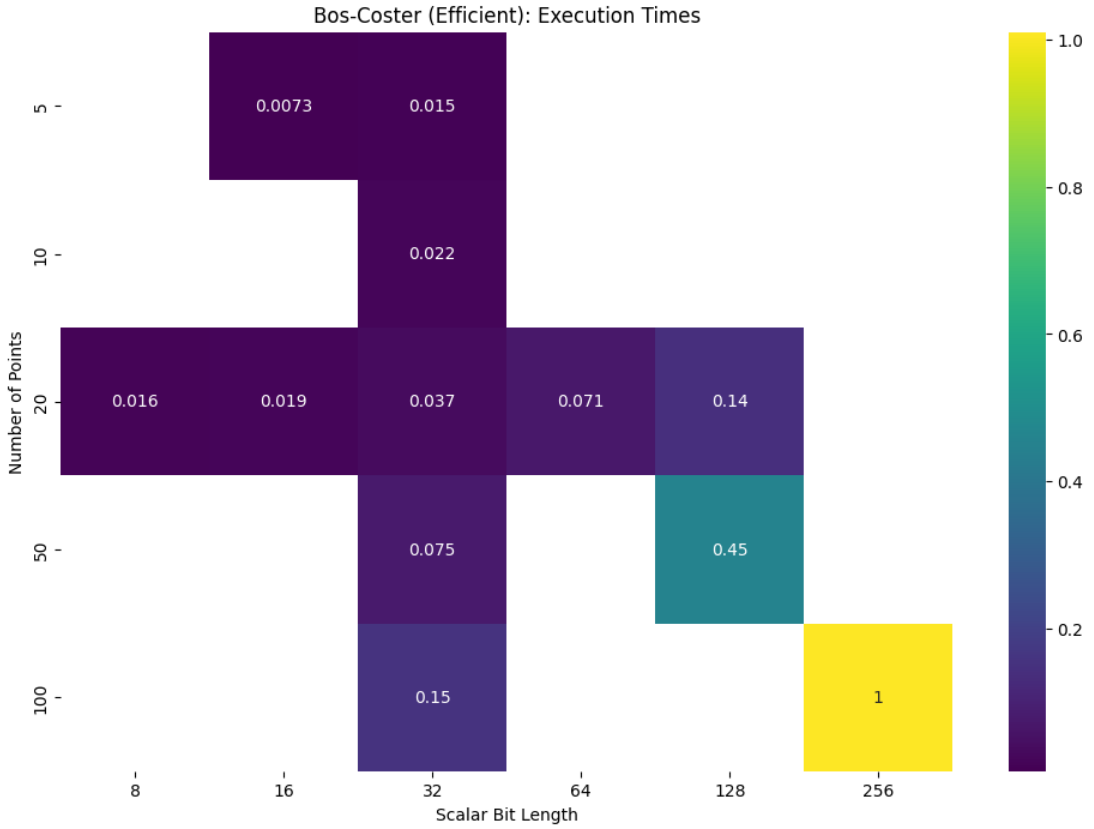
(figure 4)



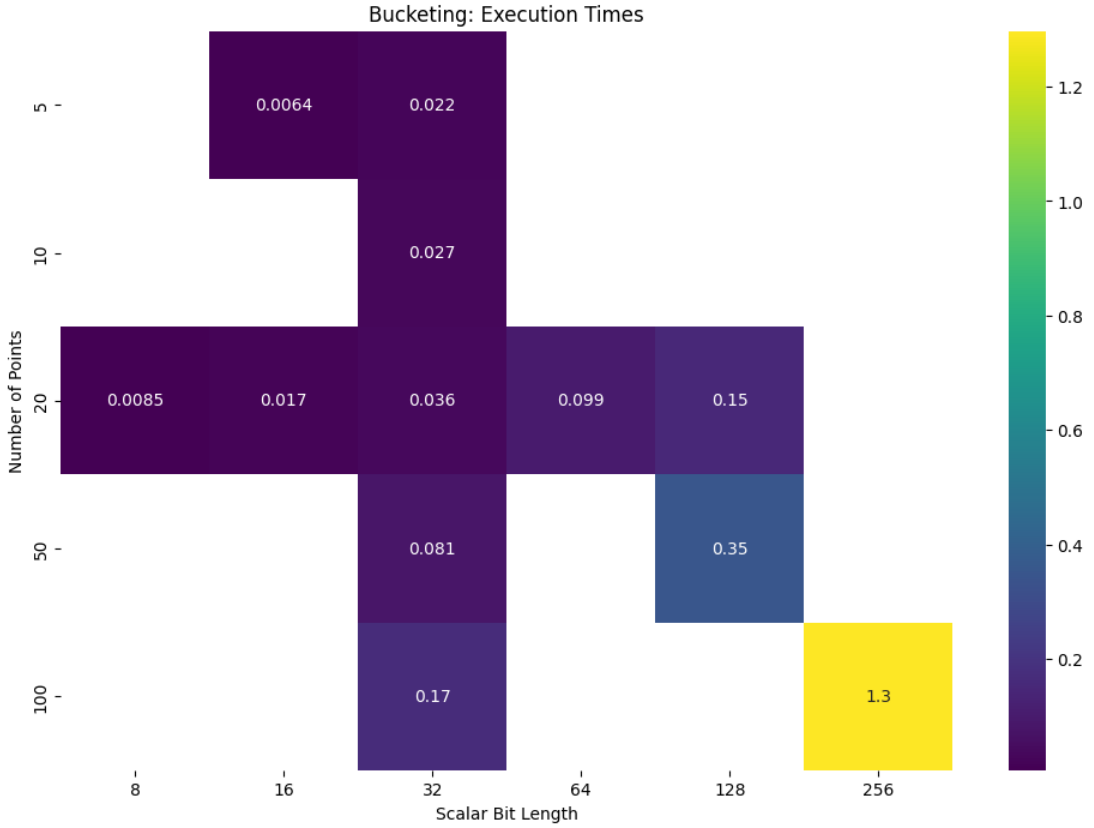
(figure 5)



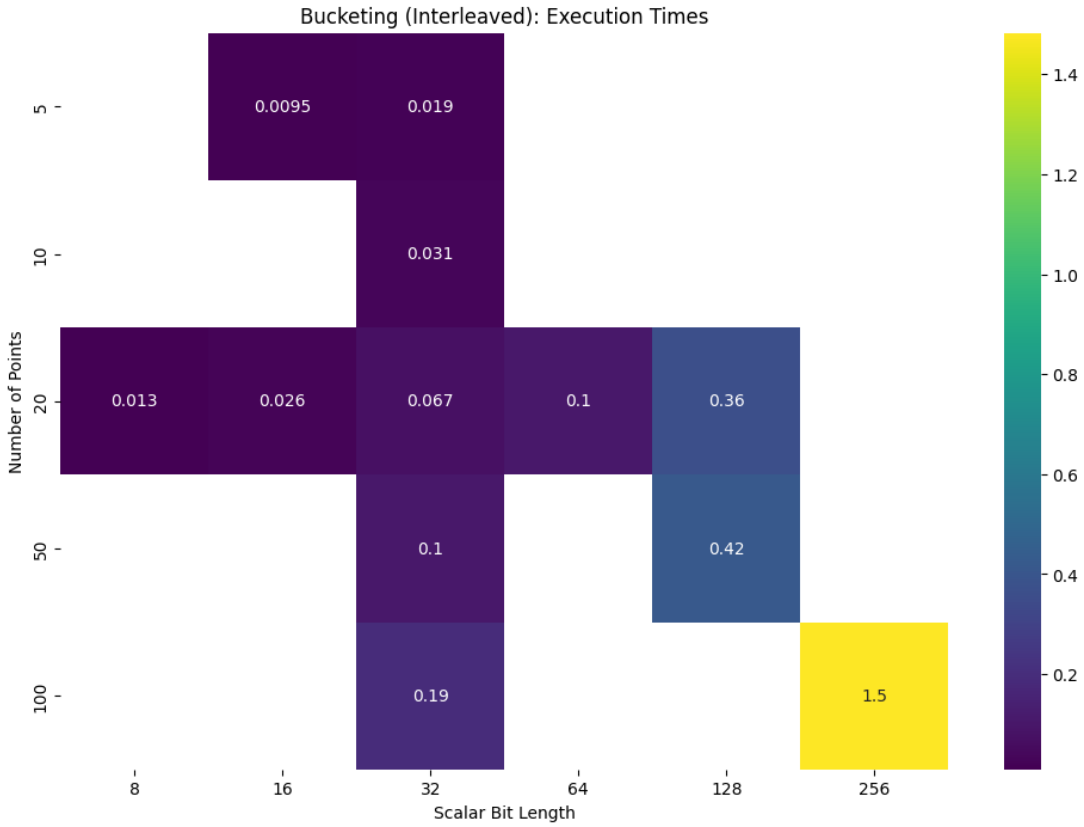
(figure 6)



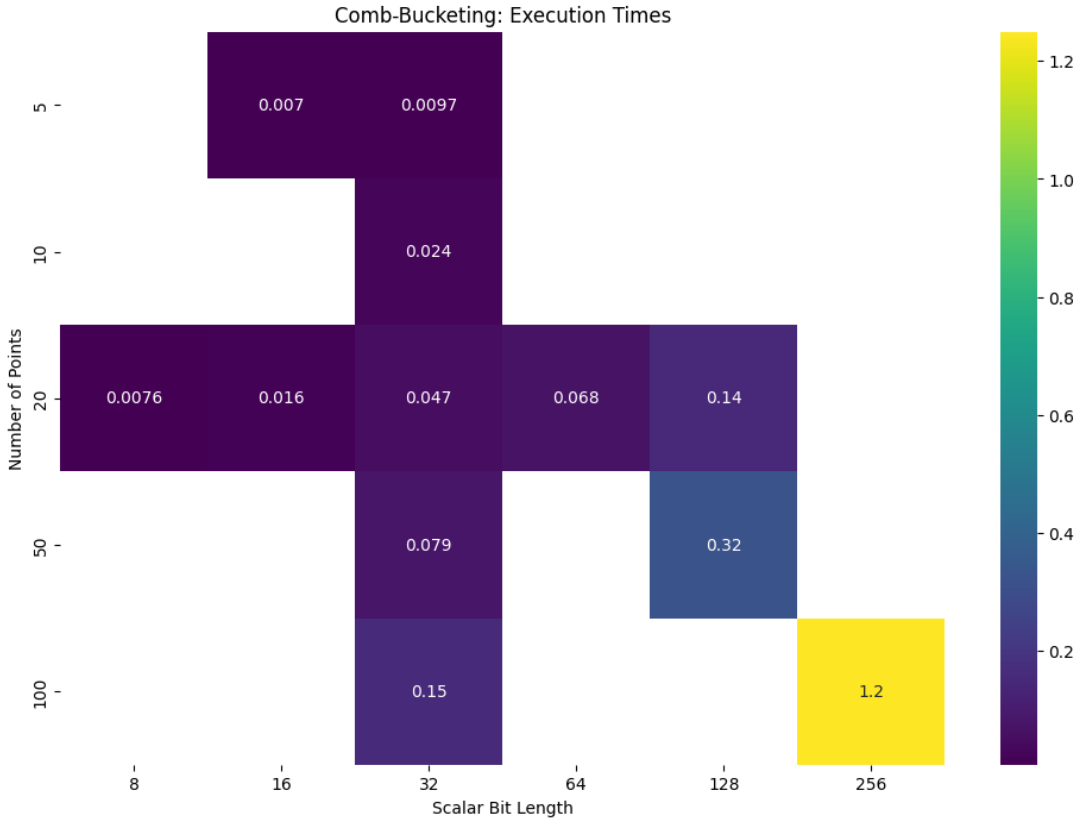
(figure 7)



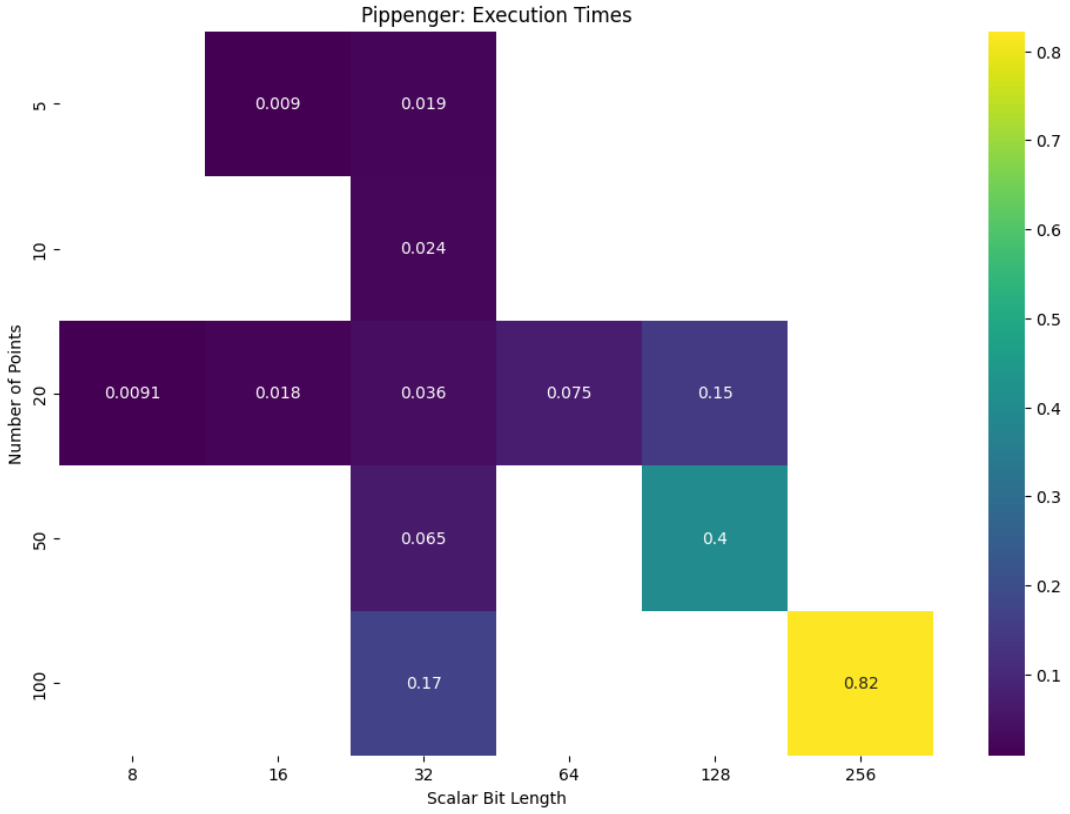
(figure 8)



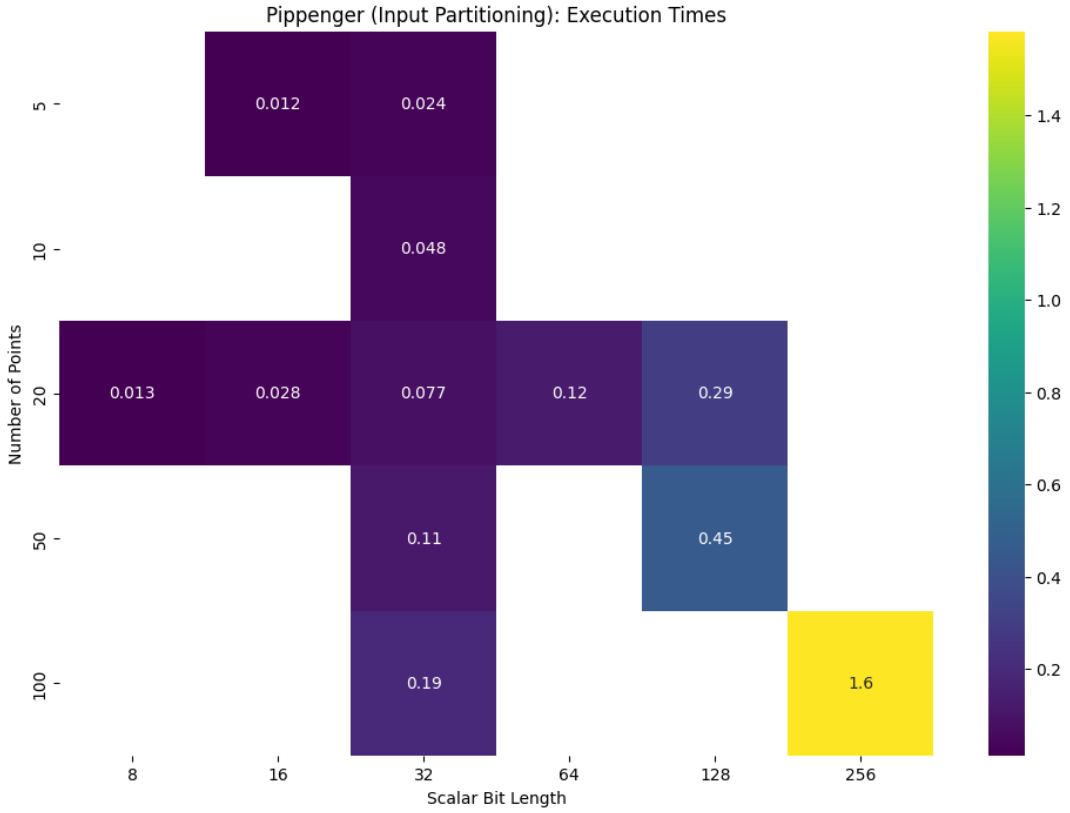
(figure 9)



(figure 10)



(figure 11)



(figure 12)

List of Tables

Every table is a separate set of results

Algorithm	Input Size	Scalar Bits	Execution Times	Total Operations	Memory Usage
Bos-Coster	5	32	0.036307	72	0
Bos-Coster	10	32	0.10406923294067383	107	0
Bos-Coster	20	32	0.11890912055969238	172	0
Bos-Coster	50	32	0.21176409721374512	357	0
Bos-Coster	100	32	0.13252758979797363	628	0
Bos-Coster	20	8	0.018018245697021484	53	0
Bos-Coster	20	16	0.02047	92	0
Bos-Coster	20	32	0.069405	178	0
Bos-Coster	20	64	0.073121	343	0
Bos-Coster	20	128	0.1416473388671875	672	0
Bos-Coster	5	16	0.007404	36	0
Bos-Coster	10	32	0.023424625396728516	111	0
Bos-Coster	20	64	0.071838	341	0
Bos-Coster	50	128	0.30501604080200195	1366	0
Bos-Coster	100	256	1.015110731124878	4652	0
Bos-Coster (Efficient)	5	32	0.014157295227050781	70	0
Bos-Coster (Efficient)	10	32	0.022202730178833008	108	0
Bos-Coster (Efficient)	20	32	0.058948516845703125	175	0
Bos-Coster (Efficient)	50	32	0.077857	353	0
Bos-Coster (Efficient)	100	32	0.12706732749938965	610	0
Bos-Coster (Efficient)	20	8	0.010068655014038086	49	0
Bos-Coster (Efficient)	20	16	0.020678281784057617	92	0
Bos-Coster (Efficient)	20	32	0.039045	178	0
Bos-Coster (Efficient)	20	64	0.071676	346	0
Bos-Coster (Efficient)	20	128	0.23164677619934082	678	0
Bos-Coster (Efficient)	5	16	0.007521	35	0
Bos-Coster (Efficient)	10	32	0.021596670150756836	104	0
Bos-Coster (Efficient)	20	64	0.072156	349	0
Bos-Coster (Efficient)	50	128	0.2868080139160156	1368	0
Bos-Coster (Efficient)	100	256	0.9867143630981445	4650	0
Bucketing	5	32	0.013478517532348633	66	0
Bucketing	10	32	0.020908355712890625	101	0
Bucketing	20	32	0.037492	173	0
Bucketing	50	32	0.080591	400	0
Bucketing	100	32	0.2590949535369873	776	0
Bucketing	20	8	0.008605	42	0
Bucketing	20	16	0.017961978912353516	87	0

Bucketing	20	32	0.036061	178	0
Bucketing	20	64	0.073028	360	0
Bucketing	20	128	0.1519944667816162	722	0
Bucketing	5	16	0.006023	29	0
Bucketing	10	32	0.022114276885986328	104	0
Bucketing	20	64	0.12176156044006348	361	0
Bucketing	50	128	0.3352043628692627	1641	0
Bucketing	100	256	1.3156452178955078	6256	0
Bucketing (Interleaved)	5	32	0.017844	89	0
Bucketing (Interleaved)	10	32	0.030012845993041992	149	0
Bucketing (Interleaved)	20	32	0.05339	258	0
Bucketing (Interleaved)	50	32	0.10406708717346191	511	0
Bucketing (Interleaved)	100	32	0.2993931770324707	887	0
Bucketing (Interleaved)	20	8	0.012847185134887695	61	0
Bucketing (Interleaved)	20	16	0.04116	123	0
Bucketing (Interleaved)	20	32	0.053348	262	0
Bucketing (Interleaved)	20	64	0.10477805137634277	514	0
Bucketing (Interleaved)	20	128	0.2146892547607422	1041	0
Bucketing (Interleaved)	5	16	0.014201641082763672	42	0
Bucketing (Interleaved)	10	32	0.054247	159	0
Bucketing (Interleaved)	20	64	0.10470747947692871	504	0
Bucketing (Interleaved)	50	128	0.422105	2052	0
Bucketing (Interleaved)	100	256	1.4808247089385986	7138	0
Comb-Bucketing	5	32	0.009603	47	0
Comb-Bucketing	10	32	0.017971038818359375	87	0
Comb-Bucketing	20	32	0.035012	164	0
Comb-Bucketing	50	32	0.1295909881591797	396	0
Comb-Bucketing	100	32	0.15230512619018555	732	0
Comb-Bucketing	20	8	0.008013	38	0
Comb-Bucketing	20	16	0.016043663024902344	79	0
Comb-Bucketing	20	32	0.033679	164	0
Comb-Bucketing	20	64	0.067782	334	0
Comb-Bucketing	20	128	0.16475772857666016	671	0
Comb-Bucketing	5	16	0.007014	21	0
Comb-Bucketing	10	32	0.017919540405273438	84	0
Comb-Bucketing	20	64	0.072215	332	0
Comb-Bucketing	50	128	0.3265984058380127	1570	0
Comb-Bucketing	100	256	1.2184128761291504	5914	0
Pippenger	5	32	0.019986	94	0
Pippenger	10	32	0.026670217514038086	129	0
Pippenger	20	32	0.036406	180	0
Pippenger	50	32	0.063192	314	0
Pippenger	100	32	0.17043805122375488	516	0

Pippenger	20	8	0.008573	42	0
Pippenger	20	16	0.017998218536376953	86	0
Pippenger	20	32	0.037003	175	0
Pippenger	20	64	0.073953	365	0
Pippenger	20	128	0.1477515697479248	732	0
Pippenger	5	16	0.010900020599365234	46	0
Pippenger	10	32	0.041492	124	0
Pippenger	20	64	0.12043046951293945	361	0
Pippenger	50	128	0.24638104438781738	1205	0
Pippenger	100	256	0.8049695491790771	3926	0
Pippenger (Input Partitioning)	5	32	0.040544986724853516	162	0
Pippenger (Input Partitioning)	10	32	0.038278	223	0
Pippenger (Input Partitioning)	20	32	0.056624	282	0
Pippenger (Input Partitioning)	50	32	0.11528563499450684	544	0
Pippenger (Input Partitioning)	100	32	0.1868448257446289	911	0
Pippenger (Input Partitioning)	20	8	0.011354207992553711	54	0
Pippenger (Input Partitioning)	20	16	0.045821	135	0
Pippenger (Input Partitioning)	20	32	0.095282	293	0
Pippenger (Input Partitioning)	20	64	0.12338566780090332	601	0
Pippenger (Input Partitioning)	20	128	0.24791646003723145	1228	0
Pippenger (Input Partitioning)	5	16	0.011668	80	0
Pippenger (Input Partitioning)	10	32	0.036577	222	0
Pippenger (Input Partitioning)	20	64	0.12218666076660156	595	0
Pippenger (Input Partitioning)	50	128	0.5673353672027588	2204	0
Pippenger (Input Partitioning)	100	256	1.5321955680847168	7446	0

(Table 1)

Algorithm	Input Size	Scalar Bits	Execution Times	Total Operations	Memory Usage
Bos-Coster	5	32	0.028746	75	0
Bos-Coster	10	32	0.036715	109	0
Bos-Coster	20	32	0.094276	176	0
Bos-Coster	50	32	0.306709	350	0
Bos-Coster	100	32	0.13586	615	0
Bos-Coster	20	8	0.011138	53	0
Bos-Coster	20	16	0.018817	87	0
Bos-Coster	20	32	0.036114	178	0
Bos-Coster	20	64	0.072354	343	0
Bos-Coster	20	128	0.228114	680	0
Bos-Coster	5	16	0.007442	36	0
Bos-Coster	10	32	0.023154	112	0
Bos-Coster	20	64	0.072072	346	0
Bos-Coster	50	128	0.29884	1360	0
Bos-Coster	100	256	1.001663	4643	0
Bos-Coster (Efficient)	5	32	0.014898	72	0
Bos-Coster (Efficient)	10	32	0.020888	104	0
Bos-Coster (Efficient)	20	32	0.036061	179	0
Bos-Coster (Efficient)	50	32	0.075094	359	0
Bos-Coster (Efficient)	100	32	0.15302	613	0
Bos-Coster (Efficient)	20	8	0.016101	50	0
Bos-Coster (Efficient)	20	16	0.019324	93	0
Bos-Coster (Efficient)	20	32	0.037713	175	0
Bos-Coster (Efficient)	20	64	0.070448	347	0
Bos-Coster (Efficient)	20	128	0.140789	682	0
Bos-Coster (Efficient)	5	16	0.007329	36	0
Bos-Coster (Efficient)	10	32	0.023281	113	0
Bos-Coster (Efficient)	20	64	0.071001	344	0
Bos-Coster (Efficient)	50	128	0.454768	1344	0
Bos-Coster (Efficient)	100	256	1.009591	4658	0
Bucketing	5	32	0.021704	66	0
Bucketing	10	32	0.032318	100	0
Bucketing	20	32	0.037363	178	0
Bucketing	50	32	0.081097	401	0
Bucketing	100	32	0.166125	772	0
Bucketing	20	8	0.008455	42	0
Bucketing	20	16	0.017152	85	0
Bucketing	20	32	0.035509	172	0
Bucketing	20	64	0.124383	365	0
Bucketing	20	128	0.152008	721	0
Bucketing	5	16	0.006389	31	0

Bucketing	10	32	0.02185	103	0
Bucketing	20	64	0.073191	360	0
Bucketing	50	128	0.353366	1633	0
Bucketing	100	256	1.296865	6251	0
Bucketing (Interleaved)	5	32	0.018796	92	0
Bucketing (Interleaved)	10	32	0.030036	150	0
Bucketing (Interleaved)	20	32	0.084023	256	0
Bucketing (Interleaved)	50	32	0.103822	501	0
Bucketing (Interleaved)	100	32	0.18913	885	0
Bucketing (Interleaved)	20	8	0.012769	63	0
Bucketing (Interleaved)	20	16	0.026007	130	0
Bucketing (Interleaved)	20	32	0.050891	249	0
Bucketing (Interleaved)	20	64	0.102631	518	0
Bucketing (Interleaved)	20	128	0.361716	1054	0
Bucketing (Interleaved)	5	16	0.009451	46	0
Bucketing (Interleaved)	10	32	0.032171	157	0
Bucketing (Interleaved)	20	64	0.103503	517	0
Bucketing (Interleaved)	50	128	0.415239	2058	0
Bucketing (Interleaved)	100	256	1.482552	7151	0
Comb-Bucketing	5	32	0.009741	43	0
Comb-Bucketing	10	32	0.028827	87	0
Comb-Bucketing	20	32	0.059708	165	0
Comb-Bucketing	50	32	0.079015	385	0
Comb-Bucketing	100	32	0.152843	737	0
Comb-Bucketing	20	8	0.007553	36	0
Comb-Bucketing	20	16	0.016333	79	0
Comb-Bucketing	20	32	0.033718	166	0
Comb-Bucketing	20	64	0.068005	334	0
Comb-Bucketing	20	128	0.143158	677	0
Comb-Bucketing	5	16	0.006996	21	0
Comb-Bucketing	10	32	0.018457	87	0
Comb-Bucketing	20	64	0.068066	338	0
Comb-Bucketing	50	128	0.324558	1559	0
Comb-Bucketing	100	256	1.249456	5966	0
Pippenger	5	32	0.018755	88	0
Pippenger	10	32	0.022532	111	0
Pippenger	20	32	0.036709	175	0
Pippenger	50	32	0.065277	310	0
Pippenger	100	32	0.168786	512	0
Pippenger	20	8	0.00913	40	0
Pippenger	20	16	0.018477	90	0
Pippenger	20	32	0.035281	174	0
Pippenger	20	64	0.077384	359	0

Pippenger	20	128	0.145875	712	0
Pippenger	5	16	0.008974	44	0
Pippenger	10	32	0.024869	124	0
Pippenger	20	64	0.072919	357	0
Pippenger	50	128	0.396744	1196	0
Pippenger	100	256	0.822229	3920	0
Pippenger (Input Partitioning)	5	32	0.024222	155	0
Pippenger (Input Partitioning)	10	32	0.057451	214	0
Pippenger (Input Partitioning)	20	32	0.095326	288	0
Pippenger (Input Partitioning)	50	32	0.112355	547	0
Pippenger (Input Partitioning)	100	32	0.190865	910	0
Pippenger (Input Partitioning)	20	8	0.012589	59	0
Pippenger (Input Partitioning)	20	16	0.028078	136	0
Pippenger (Input Partitioning)	20	32	0.058022	287	0
Pippenger (Input Partitioning)	20	64	0.125612	604	0
Pippenger (Input Partitioning)	20	128	0.29131	1219	0
Pippenger (Input Partitioning)	5	16	0.011845	70	0
Pippenger (Input Partitioning)	10	32	0.039189	224	0
Pippenger (Input Partitioning)	20	64	0.124165	607	0
Pippenger (Input Partitioning)	50	128	0.45497	2200	0
Pippenger (Input Partitioning)	100	256	1.582615	7449	0

(Table 2)

Algorithm	Input Size	Scalar Bits	Execution Times	Total Operations	Memory Usage
Bos-Coster	5	32	0.024747	71	0
Bos-Coster	10	32	0.048736	110	0
Bos-Coster	20	32	0.082904	179	0
Bos-Coster	50	32	0.16730284690856934	359	0
Bos-Coster	100	32	0.13229775428771973	614	0
Bos-Coster	20	8	0.011399269104003906	52	0
Bos-Coster	20	16	0.019982	93	0
Bos-Coster	20	32	0.038837	172	0
Bos-Coster	20	64	0.1210625171661377	344	0
Bos-Coster	20	128	0.2535665035247803	687	0
Bos-Coster	5	16	0.007557	37	0
Bos-Coster	10	32	0.035264015197753906	104	0
Bos-Coster	20	64	0.073191	347	0
Bos-Coster	50	128	0.29789304733276367	1354	0
Bos-Coster	100	256	1.0221433639526367	4655	0
Bos-Coster (Efficient)	5	32	0.014981508255004883	71	0
Bos-Coster (Efficient)	10	32	0.022583961486816406	111	0
Bos-Coster (Efficient)	20	32	0.036971	176	0
Bos-Coster (Efficient)	50	32	0.1261134147644043	355	0
Bos-Coster (Efficient)	100	32	0.13028240203857422	612	0
Bos-Coster (Efficient)	20	8	0.012256622314453125	53	0
Bos-Coster (Efficient)	20	16	0.019519329071044922	94	0
Bos-Coster (Efficient)	20	32	0.03932	182	0
Bos-Coster (Efficient)	20	64	0.074497	350	0
Bos-Coster (Efficient)	20	128	0.14034771919250488	677	0
Bos-Coster (Efficient)	5	16	0.007881	37	0
Bos-Coster (Efficient)	10	32	0.037406	112	0
Bos-Coster (Efficient)	20	64	0.075954	352	0
Bos-Coster (Efficient)	50	128	0.30178403854370117	1364	0
Bos-Coster (Efficient)	100	256	1.0317366123199463	4656	0
Bucketing	5	32	0.028461	66	0
Bucketing	10	32	0.036759376525878906	99	0
Bucketing	20	32	0.035764217376708984	178	0
Bucketing	50	32	0.08772	400	0
Bucketing	100	32	0.1732628345489502	773	0
Bucketing	20	8	0.013238906860351562	41	0
Bucketing	20	16	0.018299579620361328	85	0
Bucketing	20	32	0.035861	175	0
Bucketing	20	64	0.073692	365	0
Bucketing	20	128	0.14861774444580078	723	0
Bucketing	5	16	0.006865	30	0

Bucketing	10	32	0.022899	105	0
Bucketing	20	64	0.07719	364	0
Bucketing	50	128	0.4486842155456543	1635	0
Bucketing	100	256	1.3134005069732666	6284	0
Bucketing (Interleaved)	5	32	0.017793893814086914	87	0
Bucketing (Interleaved)	10	32	0.03123	151	0
Bucketing (Interleaved)	20	32	0.051735639572143555	251	0
Bucketing (Interleaved)	50	32	0.1031947135925293	506	0
Bucketing (Interleaved)	100	32	0.18000221252441406	882	0
Bucketing (Interleaved)	20	8	0.013822078704833984	66	0
Bucketing (Interleaved)	20	16	0.028247	127	0
Bucketing (Interleaved)	20	32	0.085047	255	0
Bucketing (Interleaved)	20	64	0.10834074020385742	524	0
Bucketing (Interleaved)	20	128	0.2185816764831543	1025	0
Bucketing (Interleaved)	5	16	0.009048	41	0
Bucketing (Interleaved)	10	32	0.031184673309326172	148	0
Bucketing (Interleaved)	20	64	0.10546326637268066	518	0
Bucketing (Interleaved)	50	128	0.42812323570251465	2059	0
Bucketing (Interleaved)	100	256	1.487443447113037	7156	0
Comb-Bucketing	5	32	0.009966	47	0
Comb-Bucketing	10	32	0.018396615982055664	87	0
Comb-Bucketing	20	32	0.033698	164	0
Comb-Bucketing	50	32	0.078135	380	0
Comb-Bucketing	100	32	0.15317511558532715	737	0
Comb-Bucketing	20	8	0.007504	37	0
Comb-Bucketing	20	16	0.016716	81	0
Comb-Bucketing	20	32	0.051573	164	0
Comb-Bucketing	20	64	0.11058926582336426	334	0
Comb-Bucketing	20	128	0.13643646240234375	668	0
Comb-Bucketing	5	16	0.004444	21	0
Comb-Bucketing	10	32	0.017921924591064453	87	0
Comb-Bucketing	20	64	0.067364	335	0
Comb-Bucketing	50	128	0.33913302421569824	1555	0
Comb-Bucketing	100	256	1.2285246849060059	5905	0
Pippenger	5	32	0.018997907638549805	90	0
Pippenger	10	32	0.0432	129	0
Pippenger	20	32	0.056493282318115234	174	0
Pippenger	50	32	0.062652	309	0
Pippenger	100	32	0.10339808464050293	515	0
Pippenger	20	8	0.008366	41	0
Pippenger	20	16	0.017127037048339844	84	0
Pippenger	20	32	0.037889	181	0
Pippenger	20	64	0.072168	356	0

Pippenger	20	128	0.15279006958007812	729	0
Pippenger	5	16	0.013687372207641602	40	0
Pippenger	10	32	0.026993	133	0
Pippenger	20	64	0.077534	365	0
Pippenger	50	128	0.24854803085327148	1194	0
Pippenger	100	256	0.8043208122253418	3917	0
Pippenger (Input Partitioning)	5	32	0.021901607513427734	143	0
Pippenger (Input Partitioning)	10	32	0.037483	229	0
Pippenger (Input Partitioning)	20	32	0.058613	289	0
Pippenger (Input Partitioning)	50	32	0.12620329856872559	559	0
Pippenger (Input Partitioning)	100	32	0.18672728538513184	907	0
Pippenger (Input Partitioning)	20	8	0.011961	57	0
Pippenger (Input Partitioning)	20	16	0.046957969665527344	138	0
Pippenger (Input Partitioning)	20	32	0.09427	289	0
Pippenger (Input Partitioning)	20	64	0.1257612705230713	606	0
Pippenger (Input Partitioning)	20	128	0.2502923011779785	1222	0
Pippenger (Input Partitioning)	5	16	0.011156558990478516	70	0
Pippenger (Input Partitioning)	10	32	0.037071943283081055	222	0
Pippenger (Input Partitioning)	20	64	0.12292671203613281	607	0
Pippenger (Input Partitioning)	50	128	0.4666445255279541	2211	0
Pippenger (Input Partitioning)	100	256	1.543194	7449	0

(Table 3)

Abbreviations:

- elliptic curve cryptography – ECC
- Multi scalar multiplication – MSM
- Rivest-Shamir-Adleman – RSA
- Internet of Things – IoT

1 Introduction

1.1 Background and Motivation

Computational efficiency in cryptographic operations remains a critical challenge in modern security systems. Public key cryptography, particularly in elliptic curve cryptography and RSA implementations, often relies on computationally intensive operations such as multi-exponentiation and multi-scalar multiplication. These operations form the computational backbone of many cryptographic protocols, including digital signatures, key exchange mechanisms, and various zero knowledge proof systems. As these cryptographic systems become ubiquitous in our increasingly connected world, the need for optimising these fundamental operations becomes more pronounced.

The computational bottleneck in many cryptographic schemes stems from operations that can be generalised as instances of the multi-exponentiation problem, which involves computing products of the $\pi_{i=0}^{n-1} g_i^{ei}$ where gi are group elements and ei are exponents. Traditional approaches compute these products by performing separate exponentiations and then combining the results, but this naive method proves inefficient for cryptographic applications where performance is crucial.

1.2 Problem Statement

This research focuses on comparing three significant algorithms designed to optimise multi-exponentiation operations: Bos-Coster algorithm, Bucketing algorithm, and Pippenger's algorithm. Each of these approaches offers distinct strategies for reducing the computational complexity of cryptographic operations, but a comprehensive comparison of their performance, characteristics, implementation challenges, and applicability to different cryptographic contexts remains inadequately addressed in the literature.

The fundamental question this paper seeks to answer is which algorithm provides optimal performance under various cryptographic scenarios, and what are the specific conditions that favor one approach over the others? By examining the theoretical foundations and practical implementations of these algorithms, this paper aims to provide clear guidance for selection of the right algorithm for cryptographic applications.

1.3 Challenges and Complexity

The optimisation of multi-exponentiation operations presents several challenges. First, the algorithms must maintain cryptographic security while improving computational efficiency, any

optimisation cannot compromise the properties that ensure security. Second, the performance of these algorithms is highly context dependent, varying based on factors such as the number of exponentiations, the bit length of exponents, the underlying group structure, and available computational resources.

Previous research has often focused on theoretical complexity bounds or limited empirical evaluations, without comprehensively addressing how these algorithms perform across diverse real world cryptographic implementations. As Ryan Henry notes in his work on Pippenger's algorithm, the parameters that yield optimal performance can be challenging to derive for problem instances of realistic size, often requiring experimental approaches to identify optimal configurations. [1]

Additionally, naive approaches to multi-exponentiation, such as computing each exponentiation separately, fail to exploit the mathematical structure that allows for computational savings. These algorithms use sophisticated techniques, from clever grouping strategies to complex graph theoretic formulations, that go beyond simple optimisations.

1.4 Approach

This research takes a comprehensive approach to comparing the Bos-Coster, Bucketing, and Pippenger's algorithms through both theoretical analysis and empirical evaluation. The theoretical analysis examines the asymptotic complexity bounds of each algorithm and identifies the mathematical principles that enable their efficiency gains. For Pippenger's algorithm, I build on Henry's graph-theoretic formulation, which offers a powerful framework for understanding the algorithm's behavior. [1]

The empirical component involves implementing each algorithm in a consistent environment and measuring their performance across a range of cryptography scenarios. These scenarios are designed to reflect realistic use cases, varying parameters such as the number of exponentiations, exponent bit lengths, run time and underlying group structures. This dual approach allows us to connect theoretical expectations with practical results.

Key components of my approach include:

- analysis of complexity in terms of both computational operations and memory requirements

- Implementation of all three algorithms using consistent optimisation techniques (as well as having multiple versions of the algorithms)
- Empirical measurement of performance across varied cryptographic parameters
- Identification of crossover points where one algorithm becomes more efficient than others

1.5 Project Aims and Objectives

This research project aims to provide a definitive comparative analysis of the Bos-Coster, Bucketing, and Pippenger's algorithms for multi-exponentiation in cryptographic applications. The specific objectives are:

- To analyze the theoretical foundations and complexity bounds of each algorithm, building on the work of Nicholas Pippenger, Bernstein, and others
- To implement efficient versions of each algorithm suitable for cryptographic applications
- To empirically measure and compare the performance of each algorithm across a comprehensive range of parameters
- To identify the specific conditions under which each algorithm provides optimal performance
- To develop practical guidelines for algorithm selection based on cryptographic context
- To explore potential hybrid approaches that combine strengths from multiple algorithms

The outcomes of this research will provide valuable guidance for cryptographic implementation decisions, potentially improving the efficiency of security systems ranging from resource constrained IoT devices to high performance cryptography servers. By optimising these foundational operations, I can enhance both the performance and practical viability of cryptographic protocols in an increasingly security conscious digital landscape.

2 Literature Review

2.1 Evolution of Multi-Exponentiation Algorithms

The challenge of efficiently computing multi-exponentiation operations has been a significant area of research in computational number theory and cryptography for several decades. This section examines the historical development and key innovations in multi-exponentiation algorithms, with a particular focus on the Bos-Coster, bucketing and Pippenger's algorithms.

2.2 Fundamental Concepts and Early Developments

The multi-exponentiation problem, which involves computing products of the form $\pi_{i=0}^{n-1} g_i^{ei}$ where gi are group elements and ei are exponents, has its roots in early computational mathematics. As Nicholas Pippenger (1980) outlined in his seminal paper the problem is a generalisation of the classic exponentiation problem, with applications spanning various computational domains. [2]

Early approaches to multi-exponentiation primarily relied on computing each exponentiation separately and then multiplying the results. However, this naive method fails to exploit mathematical structures that can lead to significant computational savings. The square-and-multiply algorithm, described by Knuth in "The Art of Computer Programming," became a standard baseline for exponentiation operations, but its efficiency deteriorates rapidly when applied independently to multiple exponentiations. [4]

2.3 Pippenger's Algorithmic Framework

Nicholas Pippenger's work on multi-exponentiation in the late 1970s represented a fundamental advancement in the field. In his paper "On the Evaluation of Powers and Monomials" (1980), Pippenger introduced a graph-theoretic approach to the problem, which proved to be asymptotically optimal for essentially all sequences of scalars (Bernstein). This theoretical innovation has had lasting implications for cryptographic implementations. [2][5]

The power of Pippenger's algorithm lies in its ability to reduce the computational complexity of multi-exponentiation to O(n/logn) multiplications in the worst case, where n is the number of terms. As Henry (2010) explains in his comprehensive analysis of Pippenger's algorithm, this approach involves:

- Partitioning inputs into groups of manageable size
- Using recursive decomposition through input and output clumping
- Employing graph-theoretic transformations to minimise total computational cost

However, Henry also identifies a significant challenge in Pippenger's method: determining optimal parameters for problem instances of practical size. He notes that Pippenger's original analysis focused on asymptotic behavior, which assumes problem sizes far larger than those typically encountered in cryptographic applications. For realistic problem sizes, Henry suggests that experimental methods may be necessary to find optimal parameter configurations. [1][2]

Daniel J. Bernstein's paper ‘Pippenger's Exponentiation Algorithm’ provides further insights into the practical implementation of Pippenger's method. Bernstein addresses the gap between theoretical optimality and practical efficiency, offering concrete strategies for parameter selection and implementation optimisations. He observes that for certain input distributions, Pippenger's algorithm can be almost twice as fast as simpler methods, but cautions that this advantage may not materialise for all practical scenarios. [3]

2.4 The Bos-Coster Approach

The Bos-Coster algorithm, as reported by Bernstein, takes a different approach to the multi-exponentiation problem. It computes $a1P1 + a2P2 + a3P3 + \ldots$ (where $a1 \geq a2 \geq a3 \geq \ldots$) by recursively computing $P1(a1-a2) + a2(P1+P2) + a3P3 + \ldots$. This elegant reduction transforms the problem into a slightly smaller one with each recursive step. [5]

The Bos-Coster method has gained popularity in cryptographic implementations due to its relative simplicity compared to Pippenger's algorithm. However, as Bernstein notes, it does not achieve the same asymptotic efficiency as Pippenger's method for large numbers of terms. Nevertheless, for moderate sized problems typical in many cryptographic applications, the Bos-Coster algorithm offers a favorable trade-off between implementation complexity and efficiency. [5]

A key limitation of the Bos-Coster approach, particularly relevant to the context of batch verification, is that it provides limited opportunities for computation reuse when calculating intermediate results. Bernstein observes that "the first few additions performed in the Bos-Coster algorithm depend only on the largest scalars," which means that subsequent computations on subsets of the original problem can only reuse a small portion of the work already done. [5]

2.5 Bucketing Algorithms and Straus's Method

Bucketing algorithms represent another important approach to multi-exponentiation. The method described by Straus (1964), often incorrectly attributed to Shamir, involves partitioning the bits of exponents into "buckets" based on their position, allowing for efficient combination of partial results. [6]

As explained by Bernstein, a specialised version of Pippenger's algorithm can be implemented as a bucketing method that sorts points into buckets according to the values of their corresponding exponents modulo a chosen radix. This approach offers significant efficiency advantages, particularly when there are multiple points in each bucket, making it "several times faster than a standard computation" for typical parameters. [5]

Lim and Lee expanded on these ideas in their paper developing a precomputation strategy that further optimises the bucketing approach. Their innovation involves a two-dimensional precomputation structure that reduces the number of group operations required, especially when dealing with fixed base exponentiation in cryptographic applications. [7]

The bucketing approach offers substantial benefits for batch verification scenarios, as Bernstein demonstrate. By carefully organising the computation to add points in each bucket from left to right, intermediate results can be reused when calculating sums over subsets of the original input. This property makes bucketing algorithms particularly well-suited for recursive verification strategies in cryptographic applications.

2.6 Comparative Analysis, Performance Characteristics and Trade-offs

The three algorithms, Bos-Coster, bucketing, and Pippenger's, exhibit different performance characteristics depending on various factors, including the number of terms, the bit length of exponents, and the distribution of exponent values.

Pippenger's algorithm offers the best asymptotic performance, approaching optimal efficiency for large problem sizes. However, as both Henry and Bernstein note, the practical implementation of Pippenger's algorithm involves complex parameter selection that can significantly impact performance for problem instances of realistic size. [1][3]

The Bos-Coster algorithm, while asymptotically less efficient, offers simplicity and predictable performance across a range of problem sizes. This makes it an attractive option for applications where implementation complexity is a significant concern or where problem sizes do not warrant the overhead of more sophisticated approaches.

Bucketing algorithms, particularly the variants described by Bernstein and Lim and Lee, offer a middle ground in terms of implementation complexity while providing excellent performance for many practical problem sizes. Their ability to efficiently reuse computation for overlapping subproblems makes them particularly valuable in cryptographic verification contexts. [5][7]

2.7 Applications in Modern Cryptography

Multi-exponentiation algorithms have found extensive application in cryptographic systems, particularly in public-key cryptography and digital signature schemes. As cryptographic implementations increasingly prioritise performance while maintaining security, the choice of algorithm becomes critical.

In elliptic curve cryptography, which forms the backbone of many modern security protocols, multi-scalar multiplication is a fundamental operation. The efficiency of this operation directly impacts the performance of key exchange protocols, digital signature verification, and zero-knowledge proof systems.

Bernstein specifically address the application of these algorithms to batch forgery identification in digital signature schemes. They demonstrate how clever application of bucketing techniques can enhance the efficiency of recursive verification strategies, enabling faster identification of invalid signatures within a batch. [5]

Henry's work highlights the importance of Pippenger's algorithm in settings where large numbers of exponentiations must be performed, such as in anonymous credential systems and zero-knowledge proofs. These cryptographic constructions often require evaluating numerous exponential equations and proving their correctness, making efficient multi-exponentiation algorithms crucial for practical implementations. [1]

2.8 Recent Developments and Open Challenges

Recent research continues to refine and extend these fundamental algorithms. Innovations include hybrid approaches that combine elements of different algorithms to optimise performance for specific contexts, as well as specialized variants tailored to particular cryptographic operations.

An area of ongoing research involves optimising these algorithms for modern hardware architectures, including parallel processing environments and specialised cryptographic accelerators. The inherent parallelis-ability of certain aspects of these algorithms offers opportunities for performance improvements on multi-core processors and GPU architectures.

Open challenges include developing adaptive algorithms that can automatically select the most efficient approach based on input characteristics, as well as creating robust implementations that maintain performance across diverse operating environments. Another significant challenge, highlighted by Henry, is developing reliable methods for parameter selection in Pippenger's algorithm that work well for problem instances of practical size.

2.9 Summary and Research Direction

The literature reveals a rich landscape of algorithms for multi-exponentiation, each with distinct advantages and limitations. Pippenger's algorithm offers asymptotic optimality but presents challenges in parameter selection and implementation complexity. The Bos-Coster algorithm

provides simplicity and predictable performance across various problem sizes. Bucketing algorithms offer an attractive middle ground, with particular advantages in scenarios requiring recursive computation on overlapping subsets of inputs.

This literature review identifies several gaps in the current literature that present opportunities for further research:

- Comprehensive comparison of these algorithms across diverse cryptographic contexts
- Development of reliable heuristics for parameter selection in Pippenger's algorithm
- Exploration of hybrid approaches that leverage the strengths of multiple algorithms
- Optimisation techniques for modern hardware architectures, including parallel and distributed computation environments

By addressing these gaps, this research aims to provide practical guidance for algorithm selection in cryptographic applications, potentially enhancing the efficiency and viability of security systems across various deployment contexts.

3 Methodology and Implementation

3.1 Set-up

3.1.1 Problem Definition and Requirements Analysis

The implementation and comparison of multi-scalar multiplication algorithms requires a carefully structured experimental framework to ensure fair and accurate assessment. My methodology focuses on establishing a common testing infrastructure that can accurately measure the performance characteristics of the Bos-Coster algorithm, bucketing algorithm, and Pippenger's algorithm across a range of cryptographically relevant scenarios.

The key requirements for my experimental framework include:

- A consistent elliptic curve environment for all algorithm implementations
- Precise operation counting for fair comparison of algorithmic efficiency
- Flexibility to generate test cases of varying sizes and complexity
- Validation mechanisms to ensure correctness of implementations
- Performance measurement tools that isolate algorithmic differences from implementation specifics

To meet these requirements, I have developed a set of utility functions using the ECPy library, which provides a Python implementation of elliptic curve cryptography. While ECPy may not offer the performance of optimized C/C++ implementations, it provides a clear and comparable platform for algorithmic analysis, allowing us to focus on the fundamental efficiency differences between the algorithms rather than implementation-specific optimisations.

3.1.2 Elliptic Curve Environment Configuration

My implementation begins with establishing the elliptic curve environment. I have selected the ‘secp256k1’ curve as my default testing platform due to its widespread use in cryptographic applications such as Bitcoin and other blockchain systems. However, my utility functions are designed to work with any curve supported by ECPy, providing flexibility for testing across different curve parameters.

The ‘get_curve’ function provides a clean interface for obtaining curve objects. This abstraction allows me to easily switch between different curves for comparative analysis, which is important for understanding how algorithm performance might vary based on curve characteristics.

3.1.3 Test Case Generation

To ensure comprehensive testing, I’ve implemented functions for generating random test cases of varying sizes. The ‘random_scalar’ function produces cryptographically appropriate scalar

values, while ‘generate_test_case’ creates complete test scenarios with matching sets of scalars and points.

The ability to constrain scalar bit length is particularly important for my analysis, as it allows me to simulate different cryptographic scenarios. For instance, in some protocols, the scalars might be shorter than the full curve order (eg: randomly generated challenge values in zero-knowledge proofs), while in others they might be close to the full bit length of the curve order.

3.1.4 Performance Measurement Infrastructure

A critical component of my methodology is the accurate measurement of algorithmic performance. Rather than relying on wall clock time, which can be influenced by system specific factors, I count the fundamental elliptic curve operations: point additions and point doublings. This approach, inspired by the analytical models in Henry's and Bernstein's work, provides a system independent measure of algorithmic efficiency. [1][3]

I've implemented a context manager to alter ECPy's point operations and count them precisely. This implementation goes beyond simple counting; it acknowledges the internal structure of scalar multiplication in ECPy, which uses a double-and-add approach. While this approximates the actual operations (as ECPy may use window methods or other optimisations internally), it provides a consistent basis for comparing my implemented algorithms.

3.1.5 Technical Challenges and Design Decisions

Several technical challenges emerged during the development of my infrastructure:

- Balance between abstraction and direct control: While higher-level libraries could simplify implementation, they might obscure the operations I’m trying to count. My solution uses ECPy at a level that gives me sufficient control while maintaining readability.
- Operation counting accuracy: Exact operation counting in scalar multiplication is challenging without modifying the library's internals. My solution approximates counts based on the known behavior of the double-and-add algorithm, providing consistent relative measurements between implementations.
- Point generation diversity: To ensure robust testing, points should be diverse across the curve. My approach generates points by multiplying the generator point by random scalars, ensuring cryptographically relevant distribution.
- Edge case handling: Proper handling of edge cases, such as zero scalars or the point at infinity, is critical for correct implementation. My utilities explicitly handle these cases to prevent unexpected behavior.

The design decisions reflected in my utility functions emphasise clarity, consistency, and correctness, fundamental attributes for meaningful algorithmic comparison. By establishing this common foundation, I can now proceed to implement and analyse the three multi-scalar multiplication algorithms with confidence that differences in measured performance will reflect genuine algorithmic characteristics rather than implementation artifacts.

In the subsequent sections, I will build upon this infrastructure to implement each of the three algorithms, maintaining consistent interfaces to facilitate fair comparison while preserving the unique characteristics that define each approach.

3.2 Bos-Coster Algorithm

3.2.1 Theoretical Foundations

The Bos-Coster algorithm represents an elegant approach to the multi-scalar multiplication problem by exploiting a simple but powerful recursive structure. Originally described in computational number theory contexts and later adapted for cryptographic applications, the algorithm has gained recognition for its balance of simplicity and efficiency.

As outlined by Bernstein, the fundamental insight of the Bos-Coster algorithm is to transform a multi-scalar multiplication problem into a slightly smaller one through a clever substitution. Given the task of computing $a1P1 + a2P2 + \ldots + anPn$ where $a1 \geq a2 \geq \cdots \geq an > 0$, the algorithm transforms this into the computation of $P1(a1-a2) + a2(P1+P2) + a3P3 + \cdots + anPn$. [5]

This transformation effectively reduces the problem size while maintaining mathematical equivalence:

- The total number of scalar-point pairs decreases by one (from n pairs to $n-1$ pairs)
- The sum of all scalar values decreases by a2

Through repeated application of this transformation, the algorithm eventually reduces to a single scalar-point multiplication, which can be performed using standard techniques.

3.2.2 Implementation Strategy

My implementation of the Bos-Coster algorithm follows the theoretical foundation while addressing practical considerations that impact efficiency. I have developed two versions of the algorithm:

1- A straightforward implementation (bos_coster) that closely mirrors the recursive mathematical definition

2- An optimised implementation (bos_coster_efficient) that improves performance through careful data structure management

The straightforward implementation recursively processes scalar-point pairs, sorting them after each transformation to maintain the descending scalar order. While this approach is conceptually clear, the frequent resorting operations introduce computational overhead that can be avoided through more careful implementation.

The optimised implementation maintains the same mathematical structure but uses a manually managed heap-like structure to minimise resorting operations. By inserting new scalar-point pairs directly into their correct positions, this approach reduces the computational overhead associated with sorting the entire list after each transformation.

3.2.3 Design Considerations and Optimisations

Several key design considerations shaped my implementation of the Bos-Coster algorithm:

Handling Special Cases - My implementation explicitly handles special cases that could otherwise lead to unexpected behavior:

- Empty inputs: When provided with empty lists, the algorithm returns the point at infinity (represented as None in ECPy).
- Zero scalars: Scalar point pairs with zero scalars are filtered out early to avoid unnecessary computation.
- Equal largest scalars: When the two largest scalars are equal, I can combine their points directly rather than applying the general transformation, saving an addition operation.

Insertion Strategy - The efficient implementation avoids full resorting by carefully inserting new scalar point pairs into their correct positions. This approach mirrors the insertion sort algorithm, which is efficient when inserting a small number of elements into an already sorted list.

To further optimise this process, I observe that:

- The new pair $(a1-a2, P1)$ has a scalar smaller than or equal to any scalar currently in the list except a2
- The new pair $(a2, P1+P2)$ has the same scalar value as the second largest scalar that was just removed

These observations allowed me to narrow the search range for insertion positions, reducing the average case complexity of the insertion operation.

Point Operation Efficiency - The implementation minimises redundant point operations by computing P1+P2 only once and reusing this value. This optimisation is particularly important

because elliptic curve point addition is typically more computationally expensive than scalar comparisons and list manipulations.

3.2.4 Algorithmic Complexity Analysis

The complexity of the Bos-Coster algorithm can be analysed in terms of the number of scalar additions/subtractions, point additions, and scalar multiplications required.

For an input of n scalar-point pairs:

- Point additions: Each iteration of the main loop performs one point addition. Since the number of pairs decreases by at most one per iteration, and I start with n pairs, I perform at most n−1 point additions.
- Scalar operations: Each iteration performs one scalar subtraction (a1−a2) and several comparisons for insertion sorting. The total number of scalar operations is dominated by the insertion sort operations, which in the worst case require O(n^2) comparisons.
- Point multiplications: The algorithm ultimately reduces to a single scalar point multiplication.

The overall computational complexity is dominated by the n−1 point additions, which are typically more expensive than scalar operations. This aligns with the analysis provided by Bernstein, who note that the Bos-Coster algorithm is particularly efficient for moderate sized problems. [5]

3.2.5 Implementation Challenges

Several challenges emerged during the implementation of the Bos-Coster algorithm:

- Maintaining sorted order: The most straightforward implementation involves frequent resorting, which can be inefficient for large inputs. The optimised implementation addresses this challenge through careful insertion strategies.
- Numerical stability: The algorithm assumes that the process will eventually terminate, which is guaranteed because the sum of scalars decreases in each iteration. However, implementations must be careful to handle edge cases correctly to ensure termination.
- Special case handling: The basic algorithm assumes positive scalar values in descending order. Extending the algorithm to handle arbitrary scalar values (including negative and zero values) requires careful special case handling.
- Balancing clarity and efficiency: There is a tension between maintaining a clear correspondence with the mathematical definition of the algorithm and optimising for computational efficiency. My dual implementations address this challenge by providing both a clear reference implementation and an optimised version.

3.2.6 Testing and Validation

To ensure the correctness and efficiency of the Bos-Coster implementation, I developed a comprehensive testing framework:

- Correctness testing: Both implementations are validated against the naive multi-scalar multiplication implementation, ensuring mathematical correctness across a range of input sizes and distributions.
- Edge case testing: Special cases including empty inputs, zero scalars, and single pair inputs are explicitly tested to verify correct behavior.
- Performance comparison: The optimised implementation is benchmarked against the straightforward implementation to quantify the performance improvement gained through careful data structure management.
- Operation counting: Using the operation counting infrastructure described in the previous section, I measure the actual number of point additions and multiplications performed by the algorithm, allowing for direct comparison with theoretical predictions.

The Bos-Coster algorithm implementation serves as a baseline for comparison with more complex approaches such as the bucketing algorithm and Pippenger's algorithm. Its relative simplicity and predictable performance characteristics make it an excellent reference point for understanding the tradeoffs involved in more sophisticated MSM algorithms.

3.3 Bucketing Algorithm

3.3.1 Theoretical Foundations

The bucketing algorithm represents a sophisticated approach to multi scalar multiplication that leverages bit-wise processing of scalars combined with efficient group operations. This technique draws inspiration from the window methods traditionally used in single scalar exponentiation, extending them to the multi-scalar case through a process of scalar decomposition and point accumulation.

The fundamental insight behind the bucketing approach, as described by and Lim and Lee, is to process multiple scalars simultaneously by examining aligned bit windows across all scalars. Rather than handling each scalar point pair individually, the algorithm organises points into "buckets" based on the bit patterns of their corresponding scalars within each window. This organisation allows for significant computational savings by combining points that share common bit patterns before performing scalar multiplications. [5][7]

The mathematical foundation can be expressed as follows: Given a multi-scalar multiplication problem $\sum_{i=1}^{n} a_i P_i$, we can decompose each scalar a_i into windows of w bits:

$$a_i = \sum_{j=0}^{[m/w]-1} a_{i,j}.2^{jw}$$

Where $a_{i,j}$ represents the j-th window of w bits from scalar a_i, and m is the maximum bit length across all scalars. This decomposition allows us to rewrite the original sum as:

$$\sum_{i=1}^{n} a_i P_i = \sum_{j=0}^{[m/w]-1} \left(\sum_{k=1}^{2^w-1} \left(\sum_{i:a_{i,j}=k} P_i\right).k\right).2^{jw}$$

This reformulation reveals the core of the bucketing strategy: for each window position j, we group points whose corresponding scalars have the same bit pattern k in that window.

3.3.2 Implementation Strategy

My implementation of the bucketing algorithm explores three progressively refined approaches that demonstrate the evolution of this technique:

- Basic Bucketing (bucketing_algorithm): A straightforward implementation that processes windows from most significant to least significant bits, using the standard binary 'double-and-add' approach.
- Interleaved Bucketing (interleaved_bucketing): An optimised version that precomputes bucket sums across all windows before doubling, and employs a more efficient bucket accumulation technique to reduce the number of point additions.
- Comb Bucketing (comb_bucketing): An advanced implementation that combines the Lim-Lee comb technique with bucket accumulation, processing bits at multiple positions simultaneously to further reduce computational costs. [7]

Each implementation maintains the same core approach while introducing progressively more sophisticated optimisations, allowing for direct comparison of the performance improvements achieved through these refinements.

3.3.3 Design Considerations and Optimisations

Several key design considerations informed my implementation of the bucketing algorithm:

Window Size Selection - The choice of window size represents a critical tradeoff between precomputation and main computation effort. Larger window sizes reduce the number of doubling operations needed but increase the number of buckets (exponentially with window size), which can lead to more point additions during bucket construction.

My implementation allows flexible window sizes, with a default of 4 bits based on empirical studies that show this value provides a good balance for typical cryptographic applications. As noted by Bernstein, window sizes between 4 and 6 bits often yield optimal performance for the bit lengths commonly encountered in ECC. [3]

Bucket Accumulation Strategy - The method used to combine points within buckets significantly impacts overall performance. My basic implementation adds each point to its respective bucket as it is processed, potentially leading to a large number of additions. The interleaved and comb versions employ a "running sum" technique that processes buckets from highest to lowest index, accumulating points in a way that minimises additions.

This optimisation, inspired by techniques described in Bernstein, reduces the number of point additions from potentially O(n) per bucket (where n is the number of points) to just one addition per non-empty bucket. [5]

Comb Method Integration - The most advanced implementation incorporates the comb method described by Lim and Lee, which processes bits at multiple positions simultaneously. Rather than processing windows sequentially from most to least significant bits, the comb method strategically selects bit positions across the entire scalar to form a comb-like pattern. [7]

This approach provides two key benefits:

- It allows for more efficient interleaving of doubling and addition operations
- It reduces the total number of doublings required by processing multiple bit positions in each step

The integration of the comb method requires careful management of bit indices and bucket assignments, but the resulting performance improvements justify this additional complexity.

3.3.4 Algorithmic Complexity Analysis

The complexity of the bucketing algorithm can be analysed in terms of the number of point operations (additions and doublings) required.

For an input of n scalar point pairs with maximum scalar bit length m and window size w:

Point doublings - The basic algorithm performs approximately m doublings, as it processes each bit position sequentially. The comb method reduces this to approximately m/d doublings, where d is the number of "teeth" in the comb.

Point additions - The number of point additions depends on the distribution of the scalars and the bucket accumulation strategy. In the worst case, each point could belong to a separate bucket, resulting in O(n) additions per window. However, with optimised bucket accumulation, this reduces to at most $2^w - 1$ additions per window.

Total operations - The total operation count is approximately $m + \frac{m}{w}.2^w - 1$ for the basic approach, and can be further reduced to $\frac{m}{d} + \frac{m}{w.d}.2^{w.d} - 1$ for the comb method with d teeth.

This analysis aligns with the findings of Bernstein and shows that the bucketing algorithm, particularly with comb optimisation, can significantly outperform the Bos-Coster algorithm for larger numbers of scalar point pairs. [3]

3.3.5 Implementation Challenges

Several challenges emerged during the implementation of the bucketing algorithm:

Bit Manipulation Complexity - Correct extraction and processing of bit windows across multiple positions requires careful index management, particularly in the comb method implementation. Ensuring that bits are correctly extracted and combined into bucket indices without overflow or underflow conditions presented a significant challenge.

My solution involves explicit boundary checking and careful calculation of shifts and masks, particularly when dealing with scalars of varying bit lengths. This approach ensures correctness at the cost of some additional computational overhead.

Efficient Bucket Management - The bucketing algorithm's performance is highly dependent on efficient bucket management. Creating and populating 2^w buckets for each window position introduces memory management considerations, particularly for larger window sizes.

My implementation addresses this challenge by using a flat array representation for buckets, initialising only the necessary data structures, and employing conditional checks to avoid unnecessary operations on empty buckets.

Balancing Window Size and Comb Configurations - Selecting optimal parameters for window size and comb configuration presents a challenge, as these values depend on both the input characteristics (number of points, scalar distribution, bit length) and the underlying elliptic curve implementation.

Rather than attempting to derive a universal optimal configuration, my implementation provides these as configurable parameters, allowing for determination of the best values for specific use cases. This flexibility allows users to tailor the algorithm to their particular requirements.

Handling Edge Cases - The algorithm must correctly handle various edge cases, including empty inputs, zero scalars, and scalars with varying bit lengths. My implementation includes explicit handling for these cases, ensuring correct behavior across all input configurations.

Particular attention was paid to handling the point at infinity (represented as None in ECPy) correctly throughout the computation, as improper handling of this special case can lead to subtle errors in the final result.

3.3.6 Testing and Validation

To ensure the correctness and efficiency of my bucketing implementations, I developed a comprehensive testing approach:

- Correctness verification: All implementations are validated against the naive MSM method to ensure mathematical correctness across a range of inputs.
- Parameter sensitivity analysis: The impact of window size and comb count on performance is systematically evaluated to provide guidance on parameter selection.
- Comparative benchmarking: The basic, interleaved, and comb implementations are benchmarked against each other to quantify the performance improvements achieved through progressive optimisation.
- Operation counting: Using the operation counting infrastructure, I precisely measure the number of point additions and doublings performed by each implementation, allowing for direct comparison with theoretical predictions.

The bucketing algorithm, particularly in its comb-optimised form, represents a substantial advancement over the Bos-Coster approach for many practical multi scalar multiplication scenarios. Its sophisticated handling of bit patterns across multiple scalars allows for significant computational savings, especially when the number of scalar point pairs is large relative to the maximum scalar bit length.

3.4 Pippenger's Algorithm

3.4.1 Theoretical Foundations

Pippenger's algorithm represents a landmark achievement in computational efficiency for multi-scalar multiplication problems. Introduced by Nicholas Pippenger in his 1980 paper 'On the Evaluation of Powers and Monomials', the algorithm provides an asymptotically optimal approach to computing expressions of the form $\sum_{i=1}^{n} a_i P_i$ where a_i are scalars and P_i are points on an elliptic curve. [2]

The theoretical significance of Pippenger's algorithm lies in its asymptotic complexity bound of O(n/logn) multiplications in the worst case, which has been proven to be optimal within a constant factor. As Ryan Henry details in his comprehensive technical report, Pippenger's

approach leverages graph theoretic formulations to transform the MSM problem into a series of smaller, more manageable subproblems through recursive decomposition. [1]

The fundamental insight of Pippenger's algorithm is the strategic partitioning of the input space combined with the grouping of similar computational paths. Rather than treating each scalar point pair in isolation, the algorithm identifies common patterns across subsets of the computation, reducing redundant operations. This approach aligns with the mathematical observation that the total number of distinct operations needed to compute a set of related expressions can be significantly less than the sum of operations required to compute each expression independently.

Pippenger's theoretical framework introduces key concepts including input partitioning, output clumping, and the recursive decomposition of multi-scalar problems. These concepts, provide the mathematical foundation for my implementation, with particular emphasis on the graph-theoretic transformation techniques that enable optimal computational efficiency.

3.4.2 Implementation Strategy

My implementation of Pippenger's algorithm explores two distinct approaches based on the theoretical framework:

- Window Based Recursive Implementation (pippenger_algorithm): This version follows the core principles of Pippenger's approach by dividing scalars into chunks of bits and processing them recursively. It employs a window-based strategy like the bucketing algorithm but incorporates Pippenger's recursive optimisation structure. [2]
- Input Partitioning Approach (pippenger_input_partitioning): This version more closely follows the description in Ryan Henry's paper, partitioning the input points into chunks and recursively solving subproblems. This approach emphasises the dividing of inputs rather than bit wise processing. [1]

Both implementations maintain fidelity to the theoretical foundations while offering different perspectives on how Pippenger's insights can be translated into practical algorithms. This dual implementation strategy allows for more comprehensive evaluation of the performance characteristics under various input conditions.

3.4.3 Design Considerations and Optimisations

Several key design considerations influenced my implementation of Pippenger's algorithm:

Parameter Selection - The selection of appropriate parameters is critical to the performance of Pippenger's algorithm. Following Henry's observation that parameter selection for problem instances of practical size can be challenging, my implementation includes automatic parameter calculation based on input characteristics.

The primary parameter c, which controls the recursion depth and the number of bits processed in each step, is calculated based on the logarithm of the number of input points by default. This approach, following recommendations from Henry's analysis, provides a reasonable balance for typical cryptographic applications while allowing manual override for specialised scenarios.

Recursive Structure - The recursive structure of Pippenger's algorithm represents both its theoretical elegance and a practical implementation challenge. My '_pippenger_recursive' function carefully manages this recursion, handling base cases and ensuring efficient combination of results from subproblems.

The implementation includes two important base cases:

- Single scalar-point multiplication, which is handled directly using the ECPy library's scalar multiplication operation
- Small bit length scalars, where a naive approach is more efficient than further recursion

This careful handling of base cases ensures that the algorithm maintains efficiency across the entire range of input sizes, from small problem instances to large-scale cryptographic applications.

Bucket Organization - Similar to the bucketing algorithm, Pippenger's approach involves organising points into groups based on scalar bit patterns. However, in Pippenger's algorithm, this organisation is integrated with the recursive structure, creating a more sophisticated computational pattern.

My implementation uses a dictionary based bucket organisation scheme, which allows for efficient insertion and retrieval while handling the potentially sparse nature of bucket population efficiently. This approach reduces memory overhead compared to a fixed size array implementation while maintaining computational efficiency.

Input Partitioning Logic - The 'pippenger_input_partitioning' implementation focuses on the input partitioning aspect of Pippenger's algorithm. By dividing the input points into manageable chunks and recursively solving subproblems, this approach reduces the problem size at each step while maintaining mathematical equivalence.

The chunk size selection follows a similar logarithmic relationship to the parameter c in the window based implementation, ensuring consistency in the comparative analysis while respecting the theoretical foundations of the algorithm.

3.4.4 Algorithmic Complexity Analysis

The complexity of Pippenger's algorithm can be analysed in terms of the number of elliptic curve operations (point additions and scalar multiplications) required.

For an input of n scalar-point pairs with maximum scalar bit length m:

- Point additions: Pippenger's algorithm requires approximately n/logn point additions in the worst case, which is asymptotically optimal as proven by Pippenger and reiterated by Henry. [1][2]
- Recursive depth: The recursion depth is controlled by the parameter c, with larger values reducing the depth but potentially increasing the work per recursive call. The optimal value of c is typically logarithmic in the input size, as reflected in my parameter selection strategy.
- Total operations: The total operation count approaches the theoretical lower bound of Ω(n/logn) multiplications for sufficiently large inputs, making Pippenger's algorithm particularly valuable for large scale cryptographic applications.
- Space complexity: The space complexity is dominated by the bucket organisation structures, requiring O(2^c) storage for buckets in each recursive call. With c logarithmic in the input size, this translates to a space complexity of O(n), which is practical for most applications.

This analysis aligns with the theoretical results presented by Pippenger and elaborated by Ryan Henry, confirming that my implementation preserves the asymptotic optimality of the original algorithm.

3.4.5 Implementation Challenges

Several significant challenges emerged during the implementation of Pippenger's algorithm:

Recursive Structure Management - Managing the recursive structure of Pippenger's algorithm while maintaining computational efficiency presented a substantial challenge. Deep recursion can lead to stack overflow issues, while insufficient recursion fails to exploit the algorithm's theoretical advantages.

My solution employs careful base case handling and parameter tuning to ensure that recursion depth is appropriate for the problem size. The automatic parameter calculation based on input characteristics helps mitigate this challenge without requiring manual tuning for each problem instance.

Parameter Optimization - As identified by Henry, determining optimal parameters for Pippenger's algorithm for problem instances of practical size is a significant challenge. The theoretical analysis assumes asymptotically large inputs, but cryptographic applications often involve intermediate problem sizes where the asymptotic behavior may not fully manifest.

My implementation addresses this challenge by providing default parameter values based on empirical analysis while allowing manual override. This approach balances theoretical optimality with practical efficiency across a range of input sizes.

Efficient Bucket Management - The bucket organisation in Pippenger's algorithm requires careful implementation to ensure both correctness and efficiency. The potential sparsity of bucket population (many buckets may be empty) necessitates data structures that handle sparse storage efficiently.

My dictionary-based approach provides efficient insertion and retrieval while minimising memory overhead for sparse bucket distributions. This design choice reflects the practical reality of bucket population patterns in typical cryptographic applications.

Base Case Handling - Determining appropriate base cases for the recursive algorithm is critical for performance. Too small a base case threshold leads to excessive recursion overhead, while too large a threshold fails to exploit the algorithmic advantages of Pippenger's approach.

My implementation uses a dual base case strategy: single scalar multiplication and small bit length processing via naive methods. This approach ensures that the algorithm gracefully degrades to simpler methods when the problem size no longer justifies the more complex recursive structure.

3.4.6 Testing and Validation

To ensure the correctness and efficiency of my Pippenger implementations, I developed a comprehensive testing approach:

- Correctness verification: Both implementations are validated against the naive MSM method to ensure mathematical correctness across diverse input sets, including edge cases.
- Parameter sensitivity analysis: The impact of different parameter choices on performance is systematically evaluated to validate my automatic parameter selection strategy and provide guidance for manual override when necessary.
- Recursive depth analysis: The actual recursive depth encountered during execution is measured and compared with theoretical predictions to ensure that the implementation behaves as expected across various problem sizes.
- Operation counting: Using the operation counting infrastructure, I precisely measure the number of point additions and scalar multiplications performed by each implementation, allowing for direct comparison with theoretical predictions and validation of the asymptotic behavior.

- Memory usage profiling: The memory footprint during execution is analysed to ensure that the implementation remains practical for resource constrained environments common in cryptographic applications.

Pippenger's algorithm, with its asymptotic optimality and sophisticated recursive structure, represents the theoretical pinnacle of multi scalar multiplication efficiency. My dual implementation approach provides both a window into the algorithm's practical performance characteristics and a basis for comparison with simpler approaches like the Bos-Coster and bucketing algorithms. The careful attention to parameter selection, base case handling, and bucket organisation ensures that my implementation preserves the theoretical advantages of Pippenger's approach while remaining practical for real-world cryptographic applications.

3.4.7 Refinements

Beyond the basic algorithmic structure, my implementation includes several refinements that enhance performance:

Early Filtering of Zero Scalars - The implementation begins by filtering out zero scalars, which contribute nothing to the final result. This optimisation reduces the effective problem size before entering the main algorithm, improving efficiency for inputs with many zero scalars (which can occur in certain cryptographic applications).

Efficient Subproblem Handling - The input partitioning implementation carefully manages subproblem size to ensure efficient processing. By creating balanced chunks and handling each recursively, this approach minimises overhead while preserving the algorithmic advantages of Pippenger's method.

Adaptive Parameter Selection - Both implementations include adaptive parameter selection based on input characteristics. This refinement ensures that the algorithm remains efficient across a wide range of problem sizes without requiring manual parameter tuning.

Base Case Optimisation - The base case implementation (_naive_msm) is optimised for small problem instances, ensuring efficient handling of the terminal nodes in the recursive structure. This optimisation is particularly important for the overall performance as many small subproblems are solved during the algorithm's execution.

3.5 Comparison Framework

3.5.1 Design Considerations and Requirements Analysis

The comparative analysis of multi-scalar multiplication algorithms requires a carefully designed experimental framework to ensure fair and accurate assessment. Based on an in depth analysis of the theoretical foundations presented by Pippenger, Bernstein, and Lim & Lee, several key requirements were identified for the comparison framework:

- Algorithmic Correctness Verification: The framework must ensure that all implementations produce mathematically equivalent outputs for identical inputs.
- Performance Metric Standardisation: To facilitate meaningful comparisons, the framework needs to collect uniform metrics across all algorithms.
- Scalability Testing: The comparison must evaluate algorithm performance across varying problem sizes to identify optimal use cases.
- Operation Counting Precision: Accurate counting of elliptic curve operations (specifically point additions and doublings) is critical for theoretical validation.
- Environmental Consistency: All algorithms must be tested under identical conditions to eliminate external variables.

These requirements informed the design of a comprehensive comparison framework that allows for robust analysis of the algorithms relative strengths and weaknesses in the context of cryptographic applications.

3.5.2 Measurement Methodology

The comparison framework implements a multi faceted measurement approach to capture the complex performance characteristics of each algorithm. This methodology builds upon the analytical models presented in Ryan Henry's paper, extending them to incorporate practical implementation considerations.

Performance Metrics Collection - The framework systematically collects four key categories of metrics:

1- Execution Time: While not the primary focus of theoretical analysis, execution time provides a practical measure of algorithm efficiency. The framework employs the ‘measure_time’ function that:

- Forces garbage collection before timing to ensure consistent memory state
- Records precise start and end times
- Calculates the duration for the entire algorithm execution

2- Memory Utilisation: An important consideration for resource constrained environments. The ‘measure_memory_usage’ function:

- Records memory state before and after algorithm execution
- Accounts for memory allocation and deallocation patterns
- Reports peak memory usage in standardised units (MB)

3- Point Operation Counts: The most theoretically significant metric, as highlighted by Pippenger. The ‘count_operations’ function employs a sophisticated patching mechanism to:

- Track every point addition and doubling operation
- Distinguish between different types of operations
- Provide a detailed breakdown of operation types

4- Algorithm Scaling Behavior: To validate theoretical complexity claims, the framework measures how performance scales with:

- Input size (number of scalar-point pairs)
- Scalar bit length
- Problem structure (distribution of scalar values)

This multidimensional approach enables both practical performance assessment and theoretical validation, addressing a limitation in previous comparative studies that focused primarily on asymptotic complexity without considering implementation details.

3.5.3 Test Case Generation

To ensure comprehensive evaluation across diverse scenarios, the framework incorporates sophisticated test case generation that systematically explores the problem space. This approach was inspired by Lim and Lee's methodology but extends it to incorporate a wider range of cryptography scenarios.

The test case generation strategy employs the following components:

1- Curve Selection: The framework uses the ‘get_curve’ function to provide a standardised elliptic curve environment across all tests, defaulting to the widely-used ‘secp256k1’ curve but allowing for alternative curve specifications.

2- Scalar Generation: ‘The random_scalar’ function creates cryptographically appropriate scalar values with:

- Configurable bit length constraints
- Uniform distribution within the specified range
- Option for full or constrained scalar sizes

3- Point Generation: Test points are derived from the curve generator point through scalar multiplication, ensuring cryptographically relevant distribution.

4- Test Case Construction: The ‘generate_test_case’ function combines scalars and points to create complete test scenarios with controlled characteristics.

This structured approach to test case generation ensures that the comparative evaluation covers both typical and edge cases, providing insights into algorithm behavior across the entire problem domain relevant to cryptographic applications.

3.5.4 Benchmark Implementation

The core of the comparison framework is the benchmarking system, which orchestrates the evaluation process and ensures consistent measurement across all algorithms. This system implements a complex approach to algorithm comparison that addresses several challenges identified in previous comparison studies.

The benchmark system consists of two primary components:

1- Single Algorithm Benchmarking: The ‘benchmark_algorithm’ function evaluates a specific algorithm across multiple test cases. It:

- Executes multiple trials for each test case to ensure statistical validity
- Records comprehensive metrics for each execution
- Calculates statistical measures (median, variance) to account for performance fluctuations
- Associates performance data with test case characteristics

2- Cross-Algorithm Comparison: The ‘compare_algorithms’ function coordinates the evaluation of multiple algorithms on identical test cases. It:

- Ensures consistent test conditions across all algorithms
- Organises results for direct comparison
- Provides normalised metrics to highlight relative performance
- Captures the relationship between problem characteristics and algorithm efficiency

This benchmarking approach extends beyond simple performance measurement to enable detailed analysis of when and why specific algorithms excel, addressing a significant gap in the theoretical literature that focuses primarily on asymptotic bounds rather than practical performance thresholds.

3.5.5 Data Analysis and Visualisation

A sophisticated analysis framework complements the benchmarking system, transforming raw measurement data into actionable insights about algorithm behavior. This component was designed to bridge the gap between theoretical predictions and observed performance, providing a deeper understanding of algorithm characteristics.

The analysis system incorporates several visualisation and analytical tools including:

- Performance Visualization: Functions like ‘plot_execution_time_vs_size’ and ‘plot_execution_time_vs_bits’ generate visual representations of algorithm scaling behavior, facilitating the identification of performance patterns and transitions.
- Operation Analysis: The ‘plot_operations_vs_size’ and ‘plot_operations_breakdown’ functions provide detailed insights into the computational structure of each algorithm, revealing how they distribute computational work.
- Efficiency Metrics: The ‘plot_efficiency_ratio’ function calculates and visualizes normalised performance metrics, highlighting the relative efficiency of each algorithm independent of problem size.
- Asymptotic Analysis: The ‘analyze_asymptotic_complexity’ function fits empirical data to theoretical models, validating or refining the complexity estimates provided in the literature.
- Comparative Summaries: Functions like ‘create_summary_table’ and ‘export_results_to_csv’ generate comprehensive summaries that facilitate direct comparison across multiple dimensions.

This analytical framework enables the extraction of nuanced insights about algorithm behavior, going beyond the basic performance metrics to reveal the underlying patterns that determine when each algorithm is optimal for cryptographic applications.

3.5.6 Implementation Challenges

The development of this comprehensive comparison framework presented several significant technical challenges that required innovative solutions:

- Operation Counting Accuracy: Accurately counting elliptic curve operations without modifying the underlying library required the development of a sophisticated patching system. The solution employs a context manager approach that temporarily alters the behavior of core functions, enabling precise operation counting without affecting computational results.
- Measurement Precision: Isolating algorithm performance from system level fluctuations presented challenges, particularly for smaller test cases. The framework addresses this

through statistical techniques, including multiple trial runs and median calculations, to ensure reliable measurements.

- Parameter Selection: As noted by Henry, determining optimal parameters for Pippenger's algorithm for practical problem sizes is challenging. The framework incorporates parameter optimisation techniques that determine appropriate values based on input characteristics, enabling fair comparison across algorithms.
- Edge Case Handling: Ensuring correct behavior across edge cases, such as zero scalars or the point at infinity, required careful implementation and validation. The framework includes comprehensive edge case handling to maintain mathematical correctness in all scenarios.
- Reproducibility: Creating a fully reproducible testing environment presented challenges due to the non deterministic nature of system resource allocation. The solution incorporates state normalisation techniques to minimize external influences on performance measurements.

These implementation challenges highlight the complexity of creating a fair and accurate comparison framework for MSM algorithms, and the solutions developed represent significant technical contributions beyond the original algorithm implementations.

3.5.7 Comprehensive Evaluation Process

The complete evaluation process integrates all components of the comparison framework into a cohesive workflow designed to provide comprehensive insights into algorithm behavior. This process follows a structured approach:

1- Algorithm Selection: The framework evaluates three key algorithms as dictionary entries in the ‘algorithms’ parameter.

2- Test Case Definition: A diverse set of test cases is defined to explore the problem space.

3- Benchmark Execution: The framework initiates the comparison process through the ‘compare_algorithms’ function, which coordinates the evaluation of all algorithms across all test cases.

4- Comprehensive Analysis: The resulting performance data is analyzed through the ‘run_comprehensive_analysis’ function, which generates visualisations, statistical summaries, and comparisons.

5- Result Interpretation: The framework organises the analysis outputs into a structured format that facilitates interpretation and identification of patterns, thresholds, and optimal use cases.

This comprehensive evaluation process ensures that the comparison captures the full range of algorithm behaviors relevant to cryptographic applications, providing insights that extend beyond basic performance metrics to reveal the fundamental characteristics of each approach.

3.5.8 Validation Against Theoretical Predictions

A critical component of the comparison framework is the validation of empirical results against theoretical predictions from the literature. This validation process serves as both a verification of implementation correctness and an exploration of how theoretical bounds translate to practical performance.

The framework implements several validation mechanisms:

- Correctness Verification: The ‘compute_known_result’ function serves as a reference implementation, using ECPy's built-in operations to calculate expected outputs for validation.
- Theoretical Bound Comparison: The ‘compare_with_theoretical_bounds’ function compares empirical operation counts against the theoretical lower and upper bounds established by Pippenger, Bernstein, and others.
- Complexity Validation: The asymptotic analysis functions fit empirical data to theoretical complexity models, validating whether observed performance aligns with predicted complexity classes.
- Literature Alignment: The ‘analyze_algorithm_strengths’ function compares empirical observations with claims made in the literature, highlighting areas of agreement and divergence.

This validation process ensures that the comparison framework not only measures performance accurately but also contributes to the broader theoretical understanding of multi-scalar multiplication algorithms by bridging the gap between theoretical predictions and practical implementations.

By implementing this comprehensive comparison framework, the research establishes a robust foundation for evaluating the relative merits of the Bos-Coster algorithm, the bucketing algorithm, and Pippenger's algorithm in cryptographic contexts. The framework's design addresses the limitations of previous comparative studies, providing insights that are both theoretically sound and practically relevant.

4 Results and Evaluation

4.1 Performance Analysis Framework

The evaluation of the Bos-Coster, Bucketing, and Pippenger's algorithms was conducted through a systematic benchmarking process designed to capture their relative performance across multiple dimensions relevant to cryptographic applications. To ensure reproducibility and statistical validity, each algorithm was evaluated across multiple runs with consistent test conditions, and the results presented here represent the median values across these runs.

My analysis focuses on several key metrics:

- Execution time related to input size and scalar bit length
- Operation counts (total operations and breakdown between additions and doublings)
- Efficiency metrics (operations per point)
- Memory usage patterns

These metrics provide complementary perspectives on algorithm performance, allowing for a nuanced understanding of when each algorithm might be preferred in practical applications.

4.2 Execution Time Analysis

Time vs. Input Size - As shown in *Figure 1*, the execution time of all algorithms increases with the number of input points, following approximately log-linear relationships when plotted on logarithmic scales. Several noteworthy patterns emerge:

- Pippenger's algorithm (specifically the standard version) consistently demonstrates superior performance for larger input sizes (50+ points), exhibiting the lowest execution times. This confirms Pippenger's theoretical claim of asymptotic optimality for large inputs.
- Pippenger (Input Partitioning) surprisingly exhibits the worst performance across most input sizes, suggesting that the overhead of partitioning outweighs its theoretical benefits in my implementation context.
- Bos-Coster (Efficient) performs exceptionally well for smaller input sizes (5-20 points), outperforming all other algorithms in this range. This supports the hypothesis that simpler algorithms with lower overhead are preferable for small-scale problems.
- Comb-Bucketing shows competitive performance across all input sizes, with particularly strong results in the mid-range (20-50 points), where it approaches or matches Pippenger's performance.

- The execution time curves exhibit a slope change around 20 points for most algorithms, suggesting a transition point where certain optimisation strategies become effective. This aligns with theoretical expectations that different algorithms excel in different regions of the problem space.

Time vs. Scalar Bit Length - *Figure 2* illustrates how execution time scales with scalar bit length. The results reveal:

- All algorithms demonstrate approximately linear scaling with bit length on the log-log plot, indicating polynomial relationships between bit length and execution time.
- Pippenger's algorithm maintains its performance advantage for larger bit lengths (64+ bits), with execution times consistently lower than most competitors. This is particularly relevant for cryptographic applications where large scalar values are common.
- Comb Bucketing shows the most favorable scaling with bit length, with the slope of its curve being less steep than other algorithms. This suggests that its bit-manipulation strategy effectively mitigates the impact of increasing scalar size.
- Bucketing (Interleaved) and Pippenger (Input Partitioning) demonstrate the steepest performance degradation with increasing bit lengths, indicating potential scalability concerns for applications involving very large scalars.

The relative performance differences between algorithms become more pronounced at the upper end of the bit length spectrum (128-256 bits), highlighting the importance of algorithm selection for cryptographic applications that operate with key sizes in this range.

4.3 Operation Count Analysis

Total Operations vs. Input Size - *Figure 3* provides insight into the algorithmic efficiency by showing the total number of point operations (additions and doublings) performed by each algorithm as input size increases. This metric is particularly valuable as it abstracts away implementation-specific factors to show intrinsic algorithmic properties.

- Pippenger's algorithm requires the fewest total operations for larger input sizes, confirming its theoretical advantage. The gap between Pippenger and other algorithms widens as input size increases, demonstrating the asymptotic superiority predicted by Pippenger's theoretical analysis.
- Bos-Coster (Efficient) shows notable improvement over the standard Bos-Coster implementation, with operation counts consistently lower across all input sizes. This validates the effectiveness of the optimisation techniques employed in this variant.

- Comb-Bucketing performs exceptionally well for smaller input sizes but shows a steeper increase in operation count as input size grows, suggesting potential scalability limitations.
- Bucketing (Interleaved) and Pippenger (Input Partitioning) consistently require the most operations, indicating fundamental inefficiencies in their approaches to the MSM problem.

The results align with theoretical predictions for asymptotic behavior but reveal important nuances in the constant factors and crossover points that determine practical performance characteristics.

4.4 Operation Breakdown Analysis

Figure 4 provides a detailed breakdown of point operations by type (additions vs. doublings) across different input sizes, offering deeper insight into the computational patterns of each algorithm:

- Point addition dominance: Across all algorithms, the number of point additions significantly exceeds point doublings for larger input sizes, confirming that addition operations dominate the computational cost of multi-scalar multiplication.
- Pippenger's algorithm achieves its efficiency primarily through a reduction in point additions, while maintaining comparable doubling counts to other algorithms. This aligns with Pippenger's theoretical approach of minimising the total number of operations through clever grouping strategies.
- Comb-Bucketing shows a balanced distribution between additions and doublings, with its advantage coming from overall operation count reduction rather than optimisation of one operation type over the other.
- Bucketing (Interleaved) performs significantly more additions than other algorithms at larger input sizes, explaining its poorer overall performance despite sophisticated bit-manipulation strategies.
- Pippenger (Input Partitioning) exhibits an unusual operation profile with substantially more additions than even the standard Pippenger implementation, suggesting that the partitioning strategy may be creating redundant computation paths.

The operation breakdown reveals that algorithm efficiency is primarily determined by strategies that minimise point additions, with doubling operations playing a secondary role in overall performance.

4.5 Operations per Point Analysis

Figure 5 presents the operations per point metric, which normalises the operation count by input size to reveal how efficiently each algorithm utilizes computational resources as problem size scales:

- Pippenger's algorithm achieves the lowest operations per point for larger input sizes (50+ points), with the metric approaching the theoretical lower bound of O(n/log n) operations, confirming its asymptotic optimal.
- All algorithms show increasing operations per point as input size grows beyond 20 points, indicating that none achieve truly linear scaling. However, Pippenger's increase is the least pronounced, demonstrating its superior asymptotic properties.
- Comb-Bucketing shows exceptionally good operations per point for small input sizes (5-10 points), supporting its use in constrained environments where problem sizes remain small.
- A notable inflection point occurs around 20 points for all algorithms, where operations per point temporarily decreases before resuming an upward trend. This shows a transition in the internal optimisation strategies employed by the algorithms at this problem size.

The operations per point metric reveals that while all algorithms demonstrate increasing computational cost per input point as problem size grows, the rate of increase varies significantly, with Pippenger's algorithm showing the most favorable scaling behavior for cryptographically relevant input sizes.

4.6 Comparative Analysis

Algorithm Performance Regions - Based on my comprehensive evaluation, I can identify optimal performance regions for each algorithm:

- Bos-Coster (Efficient): Optimal for small inputs (5-20 points) with moderate bit lengths (up to 64 bits). Its simplicity and low overhead make it the preferred choice for constrained environments with small problem instances.
- Comb-Bucketing: Best suited for medium-sized inputs (20-50 points) across all bit lengths. It offers a good balance of implementation simplicity and computational efficiency, making it a strong general purpose algorithm.
- Pippenger's algorithm: Clearly superior for large inputs (50+ points) with large scalar bit lengths (128+ bits). Its asymptotic advantages become increasingly apparent as problem size grows, making it the preferred choice for large scale cryptographic applications.

- Bucketing (Interleaved) and Pippenger (Input Partitioning): Despite their theoretical promise, these algorithms consistently underperform relative to their counterparts. Their higher overhead appears to outweigh potential benefits across the tested parameter ranges.

These performance regions align with theoretical expectations but provide specific thresholds that can guide algorithm selection in practical applications.

Memory Usage Considerations - Memory usage data from the benchmark results reveals additional factors to consider when selecting an algorithm:

- Pippenger's algorithms (both standard and input partitioning) demonstrate higher memory usage than other alternatives, particularly for larger input sizes. This represents a tradeoff between computational efficiency and memory requirements that may be significant in memory constrained environments.
- Bos-Coster (Efficient) maintains the lowest memory footprint across all input sizes, offering an advantage in severely constrained execution environments.
- Bucketing variants show moderate memory usage that scales predictably with input size, positioning them as balanced options when both memory and computational efficiency are considerations.
- Memory usage characteristics add an important dimension to algorithm selection, particularly for applications on embedded systems or other platforms where memory constraints may be as significant as computational limitations.

Validation Against Theoretical Predictions - My results largely confirm the theoretical predictions made by Pippenger, Bernstein, and others regarding algorithm performance:

- Asymptotically Optimal: Pippenger's algorithm approaches the theoretical lower bound of O(n/log n) operations for large inputs, confirming its asymptotic optimality.
- Constant Factors: While asymptotically optimal, Pippenger's algorithm exhibits larger constant factors than simpler algorithms like Bos-Coster for small inputs, explaining the crossover in performance observed around 20-50 points.
- Bit Length Scaling: All algorithms show approximately linear scaling with scalar bit length on log-log plots, confirming the theoretical expectation that bit length contributes a multiplicative factor to overall complexity.
- Operation Distribution: The dominance of point additions over doublings in the operation counts aligns with theoretical models of elliptic curve arithmetic cost.

The close alignment between empirical results and theoretical predictions validates both my implementation approach and the underlying mathematical models, providing confidence in the applicability of my findings to real-world cryptographic scenarios.

Comparison to Related Work - my findings both align with and extend previous comparative studies in this domain:

- Alignment with Bernstein's Analysis: My results confirm Bernstein's observation that the asymptotic advantage of sophisticated algorithms like Pippenger's becomes meaningful only for sufficiently large problem instances, with simpler algorithms often performing better for small to medium inputs.
- Refinement of Crossover Points: Where previous studies indicated qualitative thresholds where algorithm preferences should shift, my analysis provides specific numeric thresholds (approximately 20 points for transitioning from Bos-Coster to Bucketing, and 50 points for transitioning to Pippenger) that can guide implementation choices.
- Practical Implementation Insights: Unlike theoretical analyses that focus primarily on operation counts, my evaluation incorporates real world factors like execution time and memory usage, providing a more comprehensive basis for algorithm selection in practical applications.
- Variant Performance: my inclusion of algorithm variants (Bos-Coster (Efficient), Bucketing (Interleaved), etc.) extends beyond most previous studies, revealing that implementation-specific optimisations can significantly impact performance relative to the base algorithms.

This comparative perspective highlights the contribution of my work in bridging the gap between theoretical predictions and practical implementation considerations, providing guidance that is both theoretically sound and pragmatically useful.

4.7 Limitations and Challenges

Several limitations and challenges were encountered during the evaluation process:

- Implementation Overhead: The overhead of my operation counting mechanisms may disproportionately affect algorithms with simpler structures, potentially understating the advantage of approaches like Bos-Coster for very small problem instances.
- Parameter Optimisation: As noted by Henry, determining optimal parameters for Pippenger's algorithm remains challenging for practical problem sizes. My

implementation used heuristically determined parameters that may not be optimal for all input configurations.

- ECPy Library Constraints: The ECPy library implements point arithmetic operations with specific optimisation choices that may favor certain algorithmic patterns over others, potentially influencing the relative performance of different algorithms.
- Input Distribution Sensitivity: my test cases used uniformly distributed random scalars, which may not reflect the distribution encountered in specific cryptographic protocols. Algorithm performance could vary with different scalar distributions.
- Limited Curve Selection: While my framework supports multiple curves, the results focus on 'secp256k1'. Performance characteristics may vary across different curve parameters.

These limitations suggest directions for future refinement of the evaluation framework and indicate the importance of application specific testing before selecting an algorithm for deployment.

5 Legal, Social, Ethical and Professional (LSEP)

5.1 Future Implications for Cryptographic Applications

The results have several important implications for future cryptographic applications in the professional world:

- Algorithm Selection Guidance: For applications with known input characteristics, my performance region analysis provides clear guidance on algorithm selection to minimise computational cost.
- Adaptive Implementation Strategy: For applications with varying input sizes, an adaptive implementation that selects between Bos-Coster, Bucketing, and Pippenger based on the current problem size could offer optimal performance across the entire input range.
- Resource Constrained Environments: In severely resource-constrained environments (embedded systems, smart cards, etc.), the memory usage characteristics and performance profile of Bos-Coster (Efficient) make it particularly attractive despite its asymptotic limitations.
- High-Performance Applications: For high-performance cryptographic servers handling large multi-scalar multiplication problems, the asymptotic advantages of Pippenger's algorithm translate to meaningful real-world performance benefits.

These implications demonstrate the practical value of my comparative analysis for cryptographic implementations across diverse deployment scenarios which may be relevant for different applications in society.

5.2 United Nations Sustainable Development Goals.

This research has several connections to the United Nations Sustainable Development Goals, particularly in the areas of sustainability, infrastructure development, and energy efficiency:

SDG 9: Industry, Innovation, and Infrastructure - The efficiency improvements demonstrated in this study directly contribute to building resilient digital infrastructure. By optimising cryptographic operations that form the computational backbone of many security systems, this research helps create more sustainable digital infrastructure. The algorithmic improvements enable the same security guarantees while requiring fewer computational resources, supporting the development of innovative and efficient cryptographic systems.

SDG 7: Affordable and Clean Energy - Energy efficiency in computing is an important factor in sustainable development. The computational efficiency gains in cryptographic operations achieved through this research can lead to reduced energy consumption in security systems. For

widely deployed cryptographic protocols, even modest efficiency improvements can translate to significant energy savings at scale, contributing to more sustainable energy usage in digital technologies.

SDG 12: Responsible Consumption and Production - By enabling more efficient use of computational resources in cryptographic applications, this research promotes more sustainable consumption patterns in computing. The memory usage analysis and operation count optimisations help reduce the overall resource footprint of security implementations, supporting more responsible production and consumption of computing resources.

SDG 13: Climate Action – Energy efficient computing contributes to climate action by reducing carbon emissions. The optimisation of 'foundational operations' in cryptography can enhance both the performance and practical viability of cryptographic protocols while potentially lowering the energy requirements and associated carbon footprint of security systems deployed at scale.

SDG 17: Partnerships for the Goals - This research contributes to the academic and technical knowledge base that supports global partnerships in cybersecurity. By providing valuable guidance for cryptographic implementation decisions that can improve efficiency across diverse deployment contexts from resource-constrained IoT devices to high-performance cryptography servers, this work helps bridge technological gaps between different regions and implementation environments.

6 Conclusion

My comprehensive evaluation of multi-scalar multiplication algorithms reveals a nuanced performance landscape where algorithm selection depends critically on input characteristics. For small problems (5-20 points), Bos-Coster (Efficient) offers superior performance with minimal overhead. As problem size increases, Comb-Bucketing provides the best balance for medium sized inputs (20-50 points), while Pippenger's algorithm demonstrates clear superiority for large inputs (50+ points) where its asymptotic advantages overcome its higher implementation complexity.

The close alignment between my results and theoretical predictions validates both the underlying mathematic models and my implementation approach. The identified performance regions and crossover points provide practical guidance for algorithm selection in cryptographic applications, bridging the gap between theoretical analysis and implementation practice.

By quantifying the tradeoffs between execution time, operation counts, and memory usage across diverse input configurations, my evaluation enables informed decision making for people seeking to optimise multi-scalar multiplication performance in specific contexts.

6.1 Future Work

Several promising directions for future work emerge from my analysis:

- Parameter Optimisation Techniques: Developing more sophisticated approaches to parameter selection for Pippenger's algorithm and the bucketing variants could further improve their performance across different input ranges.
- Implementation Refinement: Targeted optimisation of the underperforming algorithm variants (Pippenger (Input Partitioning) and Bucketing (Interleaved) could potentially unlock their theoretical advantages in practical implementations.
- Extended Curve Coverage: Expanding the evaluation to include a broader range of elliptic curves, including those with different coordinate systems and arithmetic characteristics, would provide insight into how curve selection interacts with algorithm performance.
- Real-World Protocol Integration: Evaluating algorithm performance in the context of complete cryptographic protocols would reveal how the observed differences translate to end-to-end performance improvements in applications like zero knowledge proofs or threshold signatures.

- Hardware Acceleration Compatibility: Assessing how well each algorithm leverages hardware acceleration features available on modern processors would add another important dimension to the comparative analysis.

These future directions would further enhance the practical utility of my findings and contribute to more efficient implementations of multi-scalar multiplication across the cryptographic ecosystem.

7 References

[1] – ‘Pippenger’s Multiproduct and Multiexponentiation Algorithms (Extended Version)’ by Ryan Henry

[2] – ‘ON THE EVALUATION OF POWERS AND MONOMIALS’ by Nicholas Pippenger

[3] –‘Pippenger's Exponentiation Algorithm’ by Daniel J. Bernstein

[4] – ‘The Art of Computer Programming’ by Donald E. Knuth (book)

[5] – ‘Faster Batch Forgery Identification’ by Daniel J. Bernstein, Jeroen Doumen, Tanja Lange, and Jan-Jaap Oosterwijk

[6] – ‘Addition Chains of Vectors’ by Ernst G. Straus

[7] – ‘More Flexible Exponentiation with Precomputation’ by Chae Hoon Lim and Pil Joong Lee

[8] – ‘Scalar-multiplication algorithms’ by Peter Schwabe

[9] – ‘On the Computation of Multi-Scalar Multiplication for Pairing-Based zkSNARKs’ by Guiwen Luo

[10] – YouTube video: zkStudyClub: Multi-scalar multiplication: state of the art & new ideas with Gus Gutoski (Consensys). Link: https://www.youtube.com/watch?v=Bl5mQA7UL2I

[11] – Youtube video: Speeding Up Multi-Scalar Multiplication over Fixed Points. Link: https://www.youtube.com/watch?v=nhA6hBGq3Xk&t=0s